



LABORATORIS

PROGRAMACIÓ CIENTÍFICA

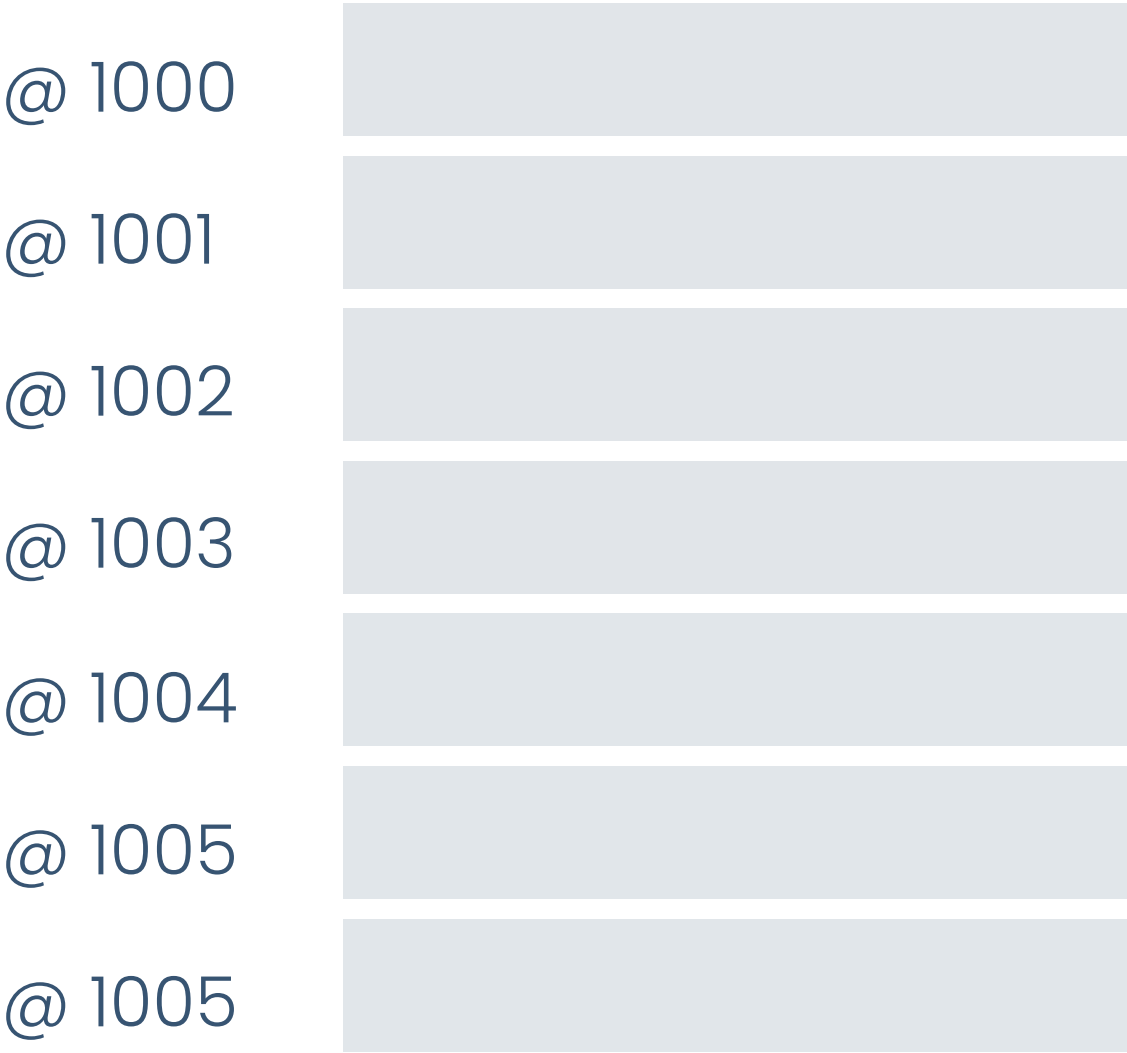
Sessió de repàs

REPASSEM...

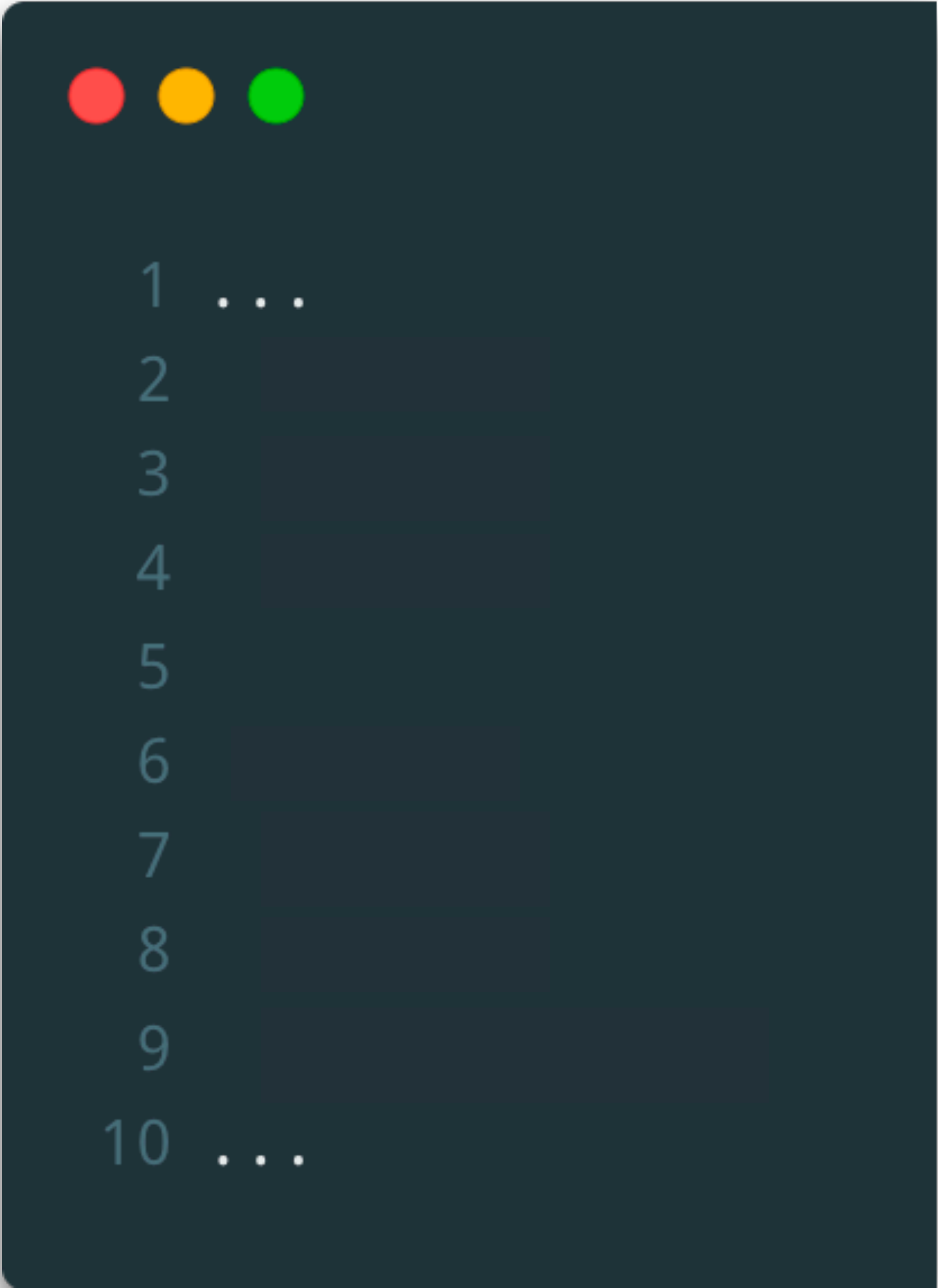
PUNTERS

Recordatori de punters

Funcionament bàsic

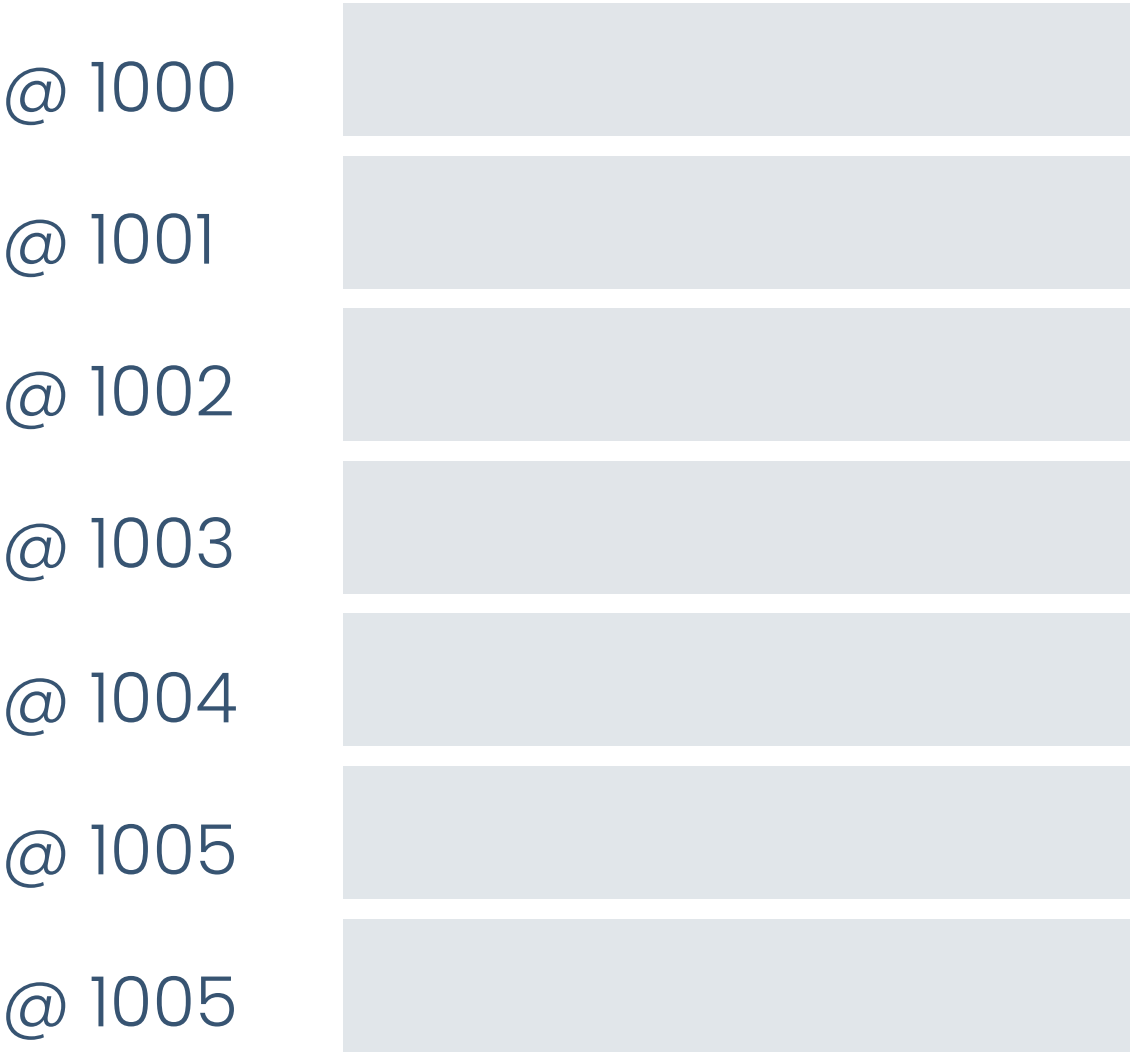


Memòria de 32 bits



Recordatori de punters

Funcionament bàsic

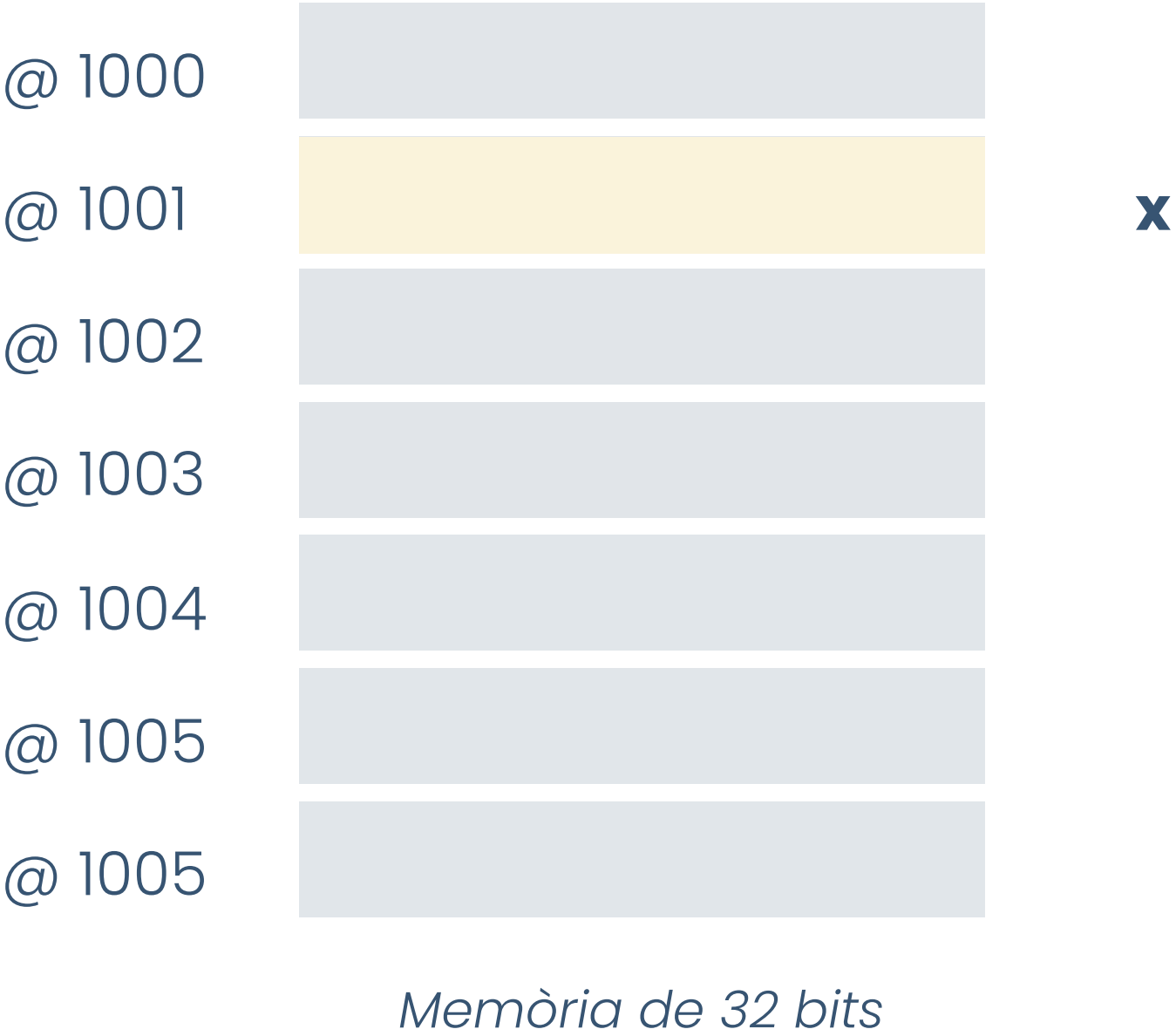


Memòria de 32 bits

```
1 ...
2   int x;
3
4
5
6
7
8
9
10 ...
```


Recordatori de punters

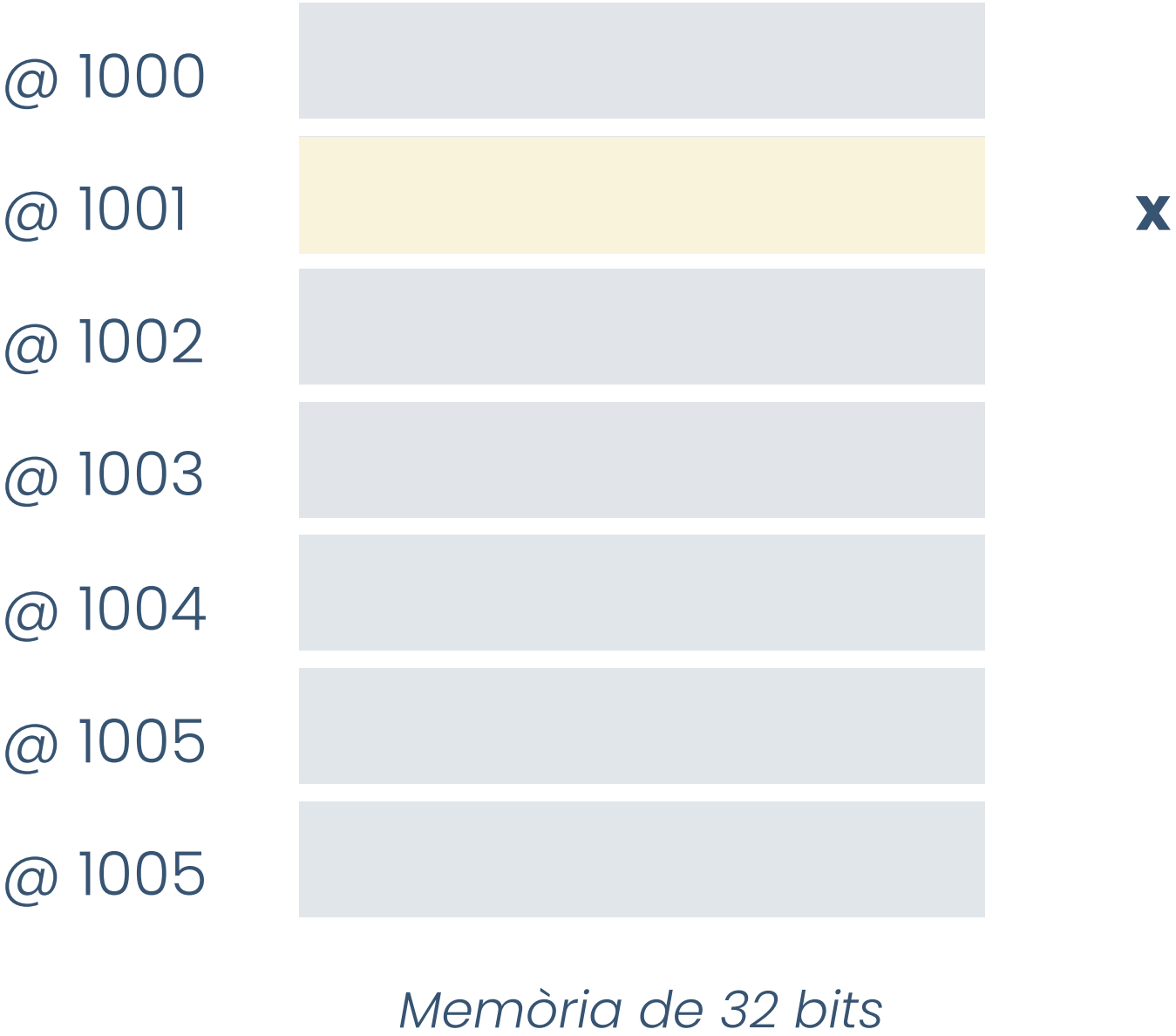
Funcionament bàsic



```
1 ...
2  int x;
3
4
5
6
7
8
9
10 ...
```

Recordatori de punters

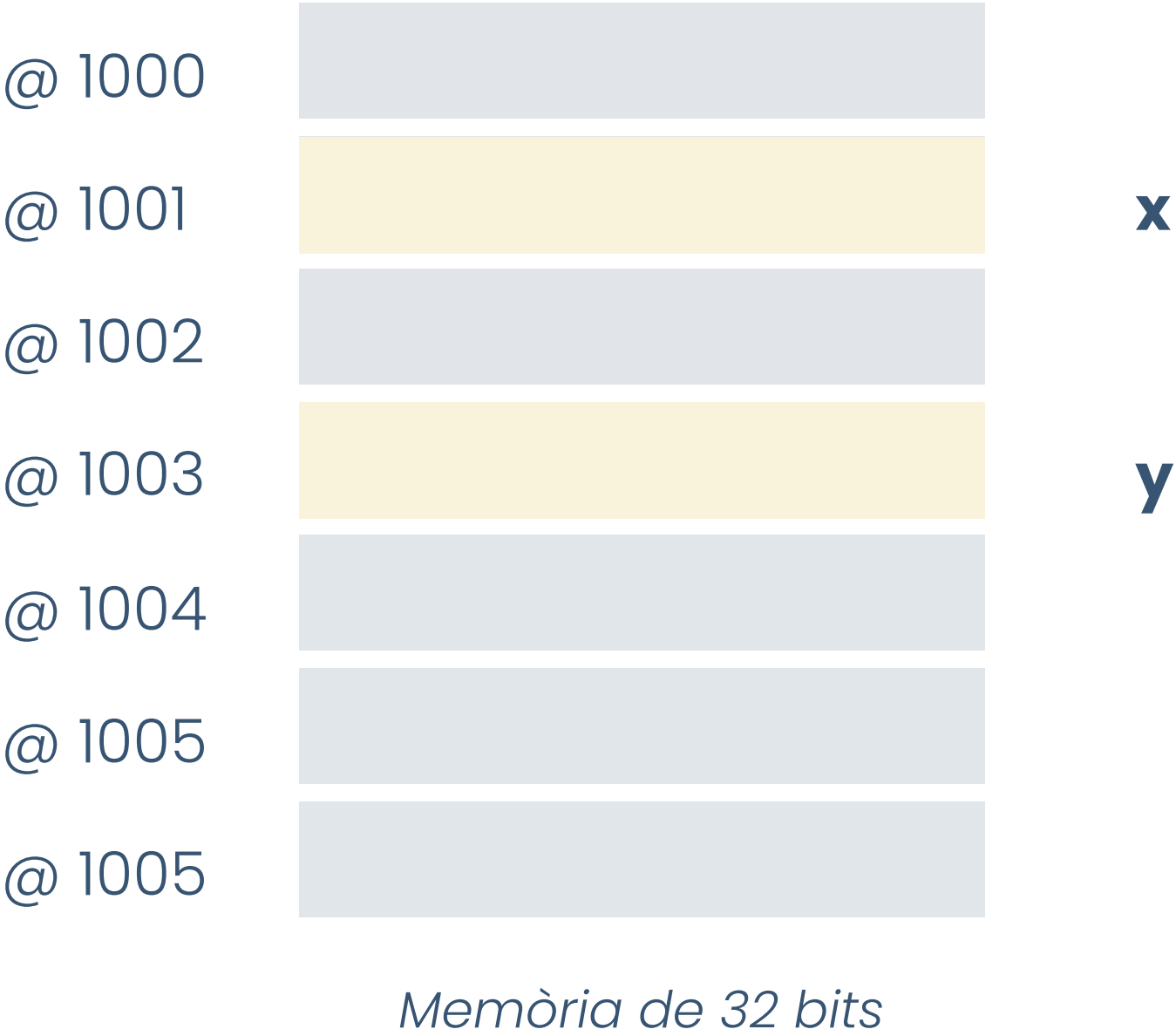
Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4
5
6
7
8
9
10 ...
```

Recordatori de punters

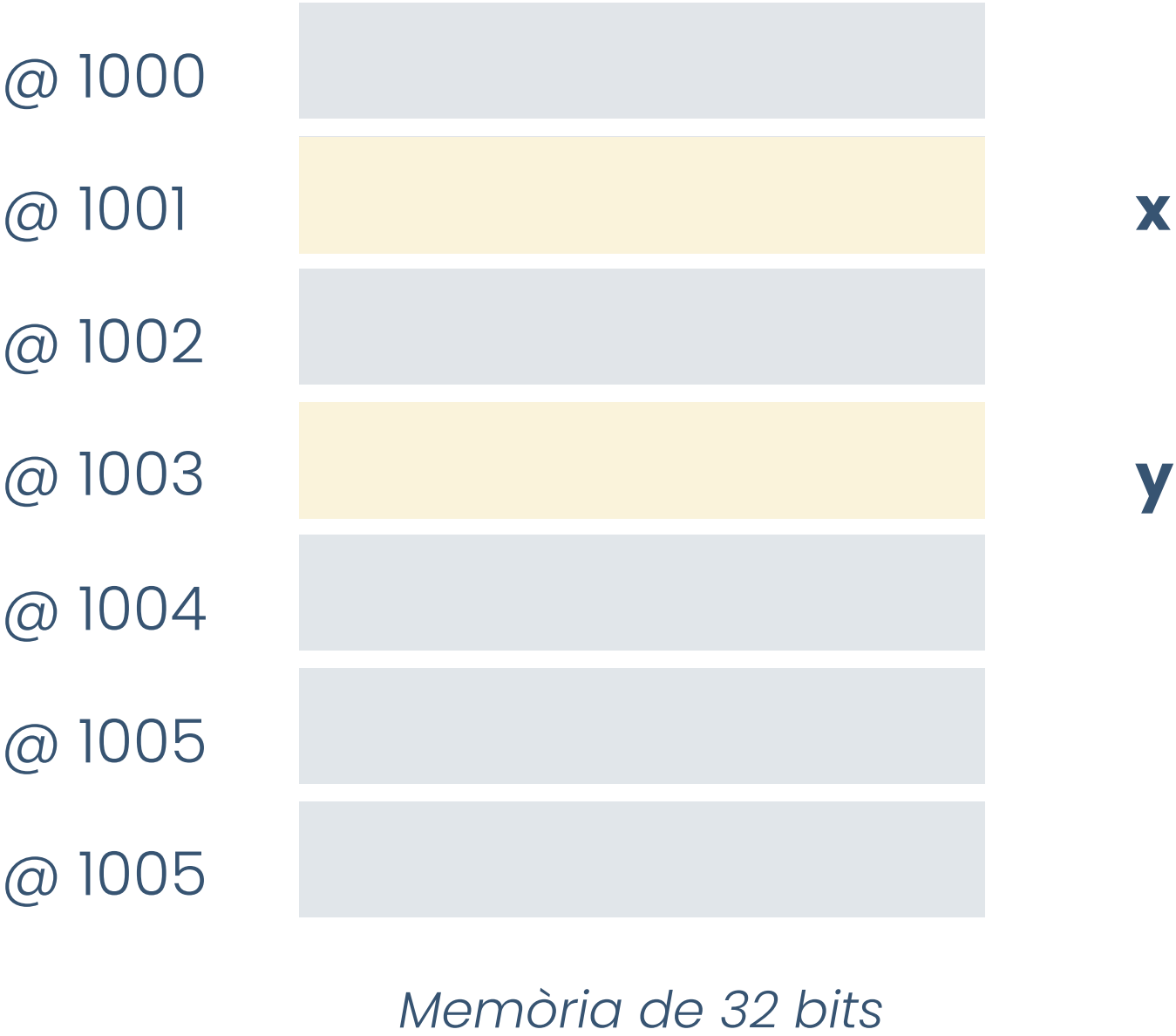
Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4
5
6
7
8
9
10 ...
```


Recordatori de punters

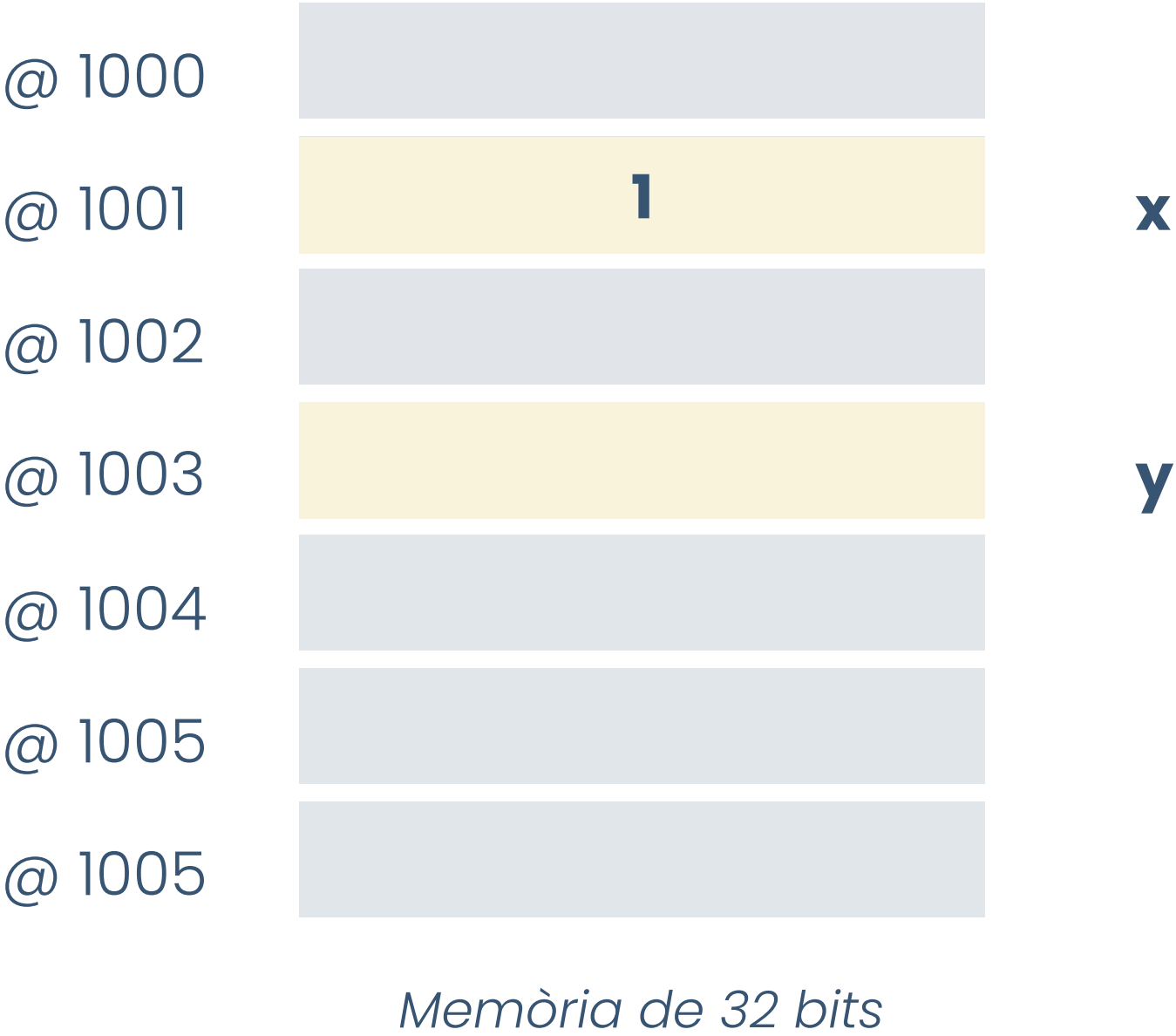
Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4
5
6  x = 1;
7
8
9
10 ...
```


Recordatori de punters

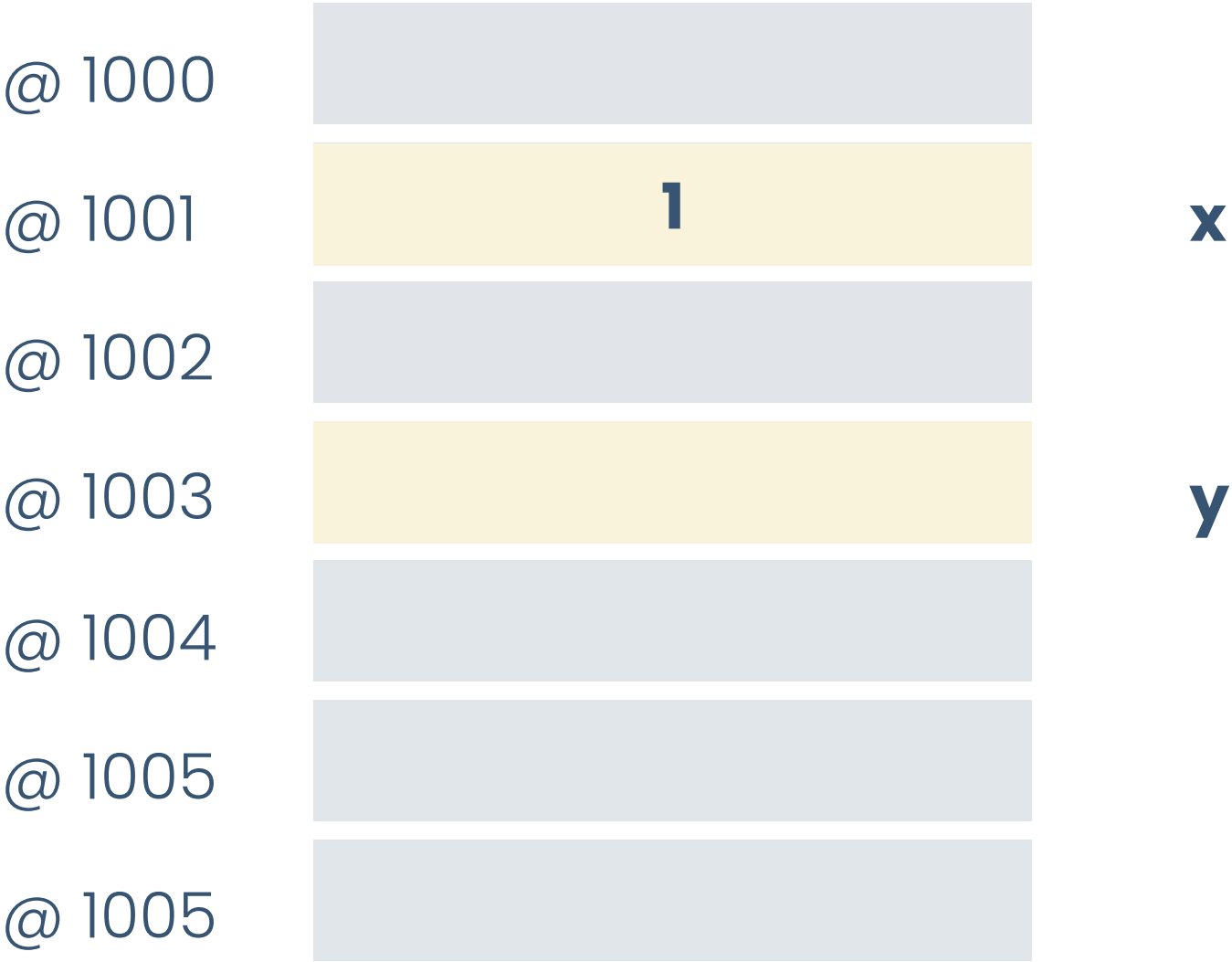
Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4
5
6  x = 1;
7
8
9
10 ...
```

Recordatori de punters

Funcionament bàsic

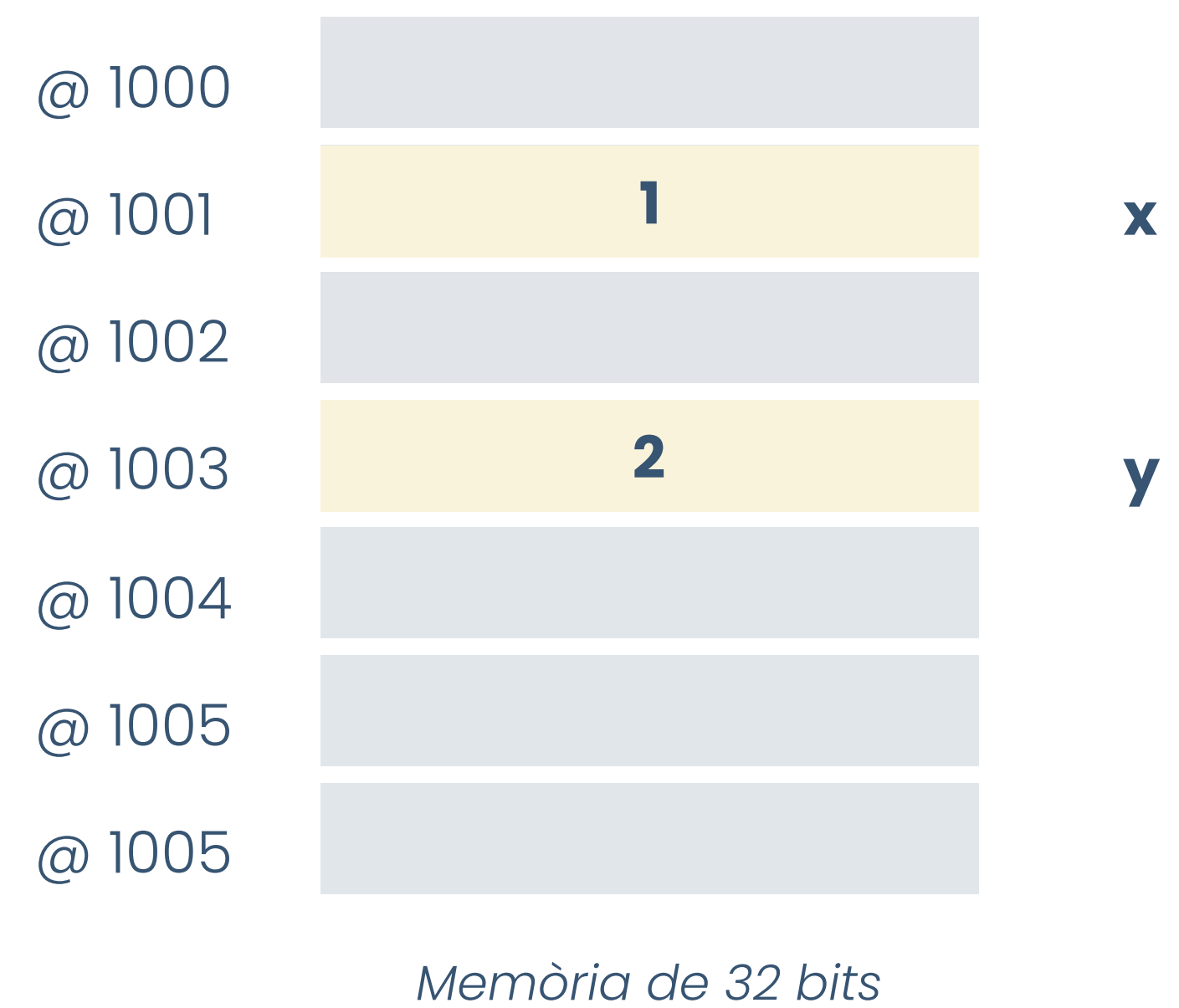


Memòria de 32 bits

```
1 ...
2  int x;
3  int y;
4
5
6  x = 1;
7  y = 2;
8
9
10 ...
```

Recordatori de punters

Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4
5
6  x = 1;
7  y = 2;
8
9
10 ...
```


Recordatori de punters

Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4
5
6  x = 1;
7  y = 2;
8
9
10 ...
```


Recordatori de punters

Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4  int *p;
5
6  x = 1;
7  y = 2;
8
9
10 ...
```

Recordatori de punters

Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4  int *p;
5
6  x = 1;
7  y = 2;
8
9
10 ...
```

Recordatori de punters

Funcionament bàsic



```
1 ...
2  int x;
3  int y;
4  int *p;
5
6  x = 1;
7  y = 2;
8
9
10 ...
```

Operador "&": et dóna l'adreça de memòria d'un objecte
Operador "*": aplicat a un apuntador, dona accés a l'objecte al qual s'apunta

Recordatori de punters

Funcionament bàsic

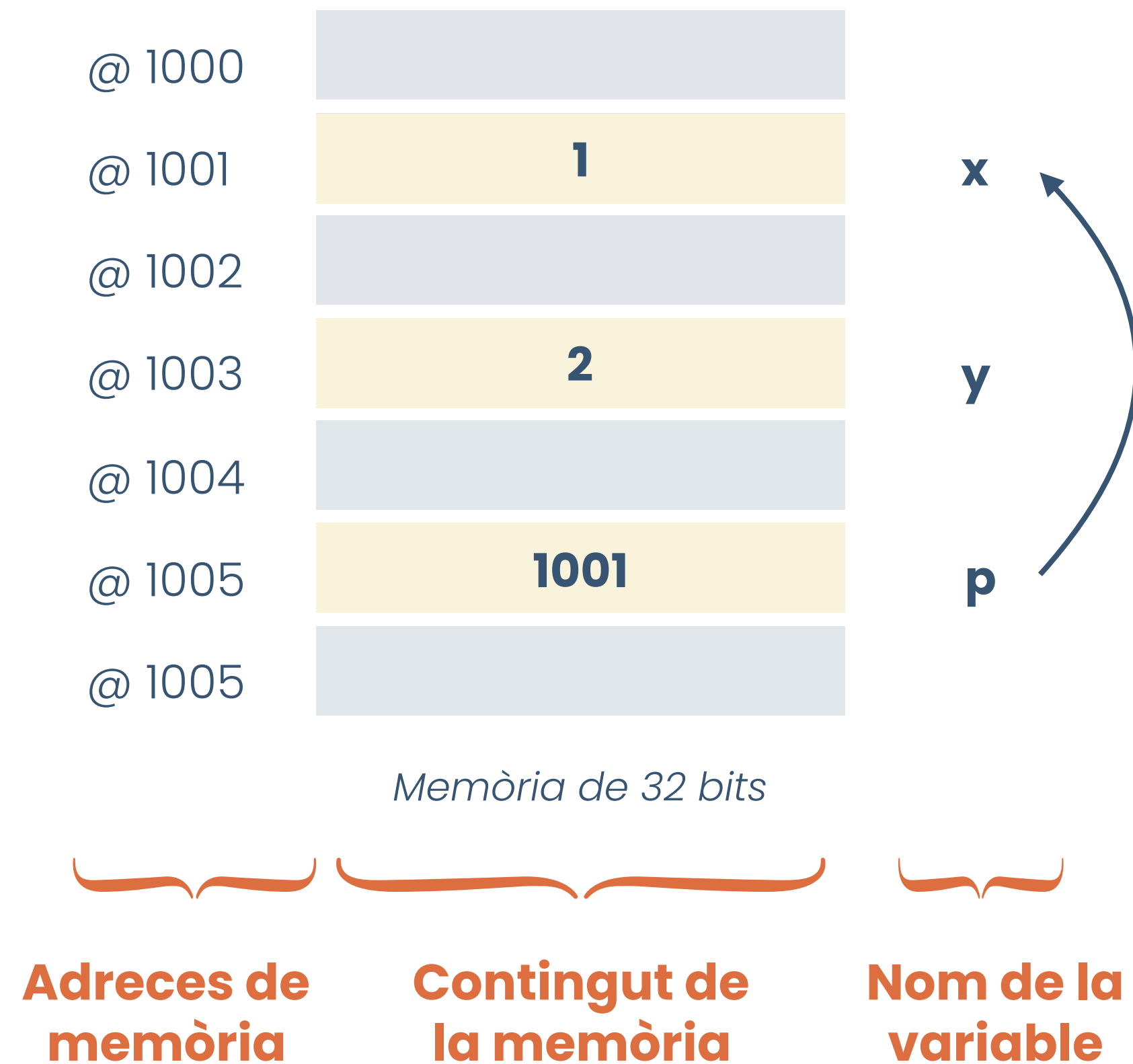


```
1 ...
2  int x;
3  int y;
4  int *p;
5
6  x = 1;
7  y = 2;
8  p = &x;
9
10 ...
```

Operador "&": et dóna l'adreça de memòria d'un objecte
Operador "*": aplicat a un apuntador, dona accés a l'objecte al qual s'apunta

Recordatori de punters

Funcionament bàsic



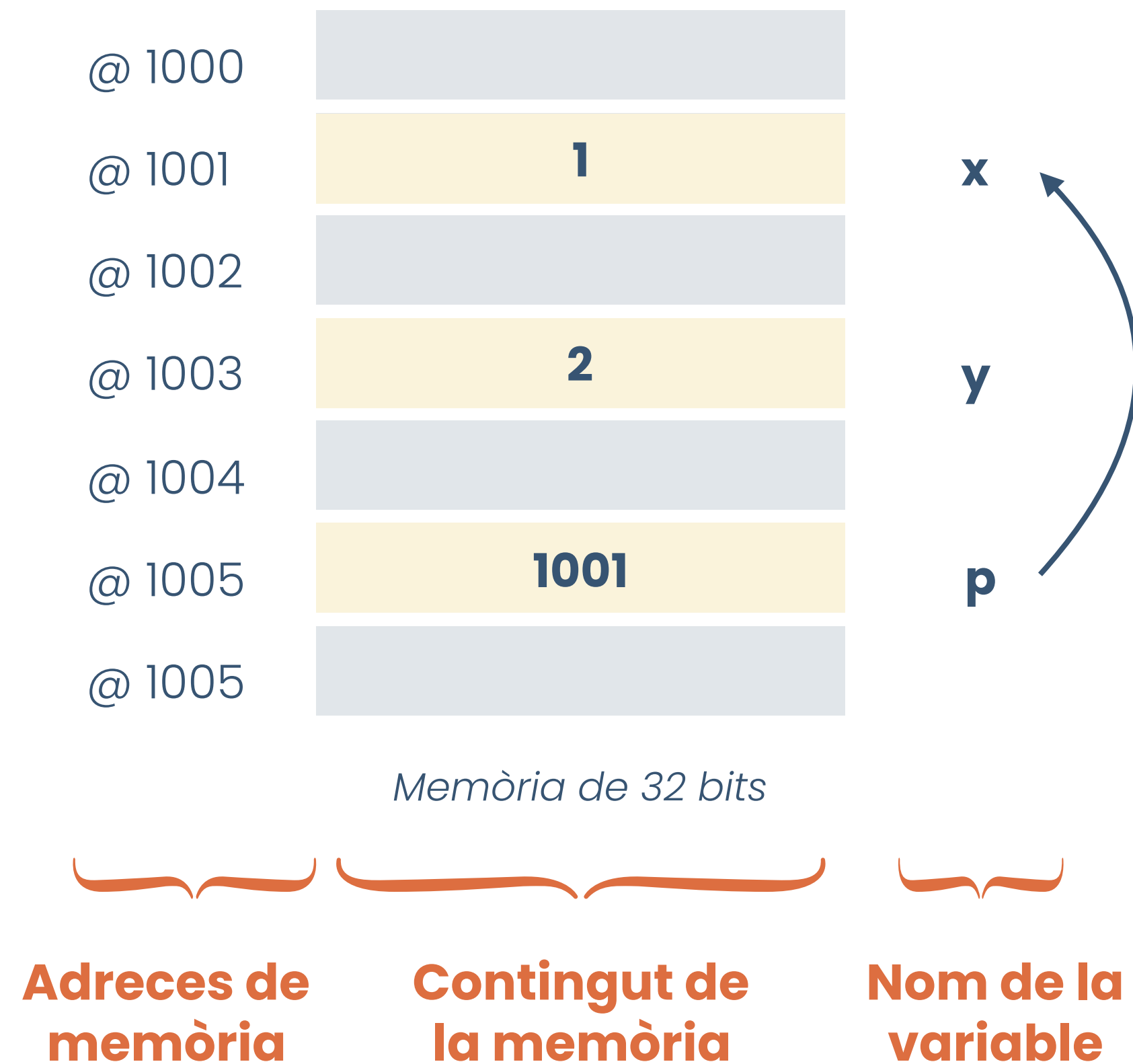
```
1 ...  
2     int x;  
3     int y;  
4     int *p;  
5  
6     x = 1;  
7     y = 2;  
8     p = &x;  
9  
10 ...
```

Operador "&": et dóna l'adreça de memòria d'un objecte

Operador "*": aplicat a un apuntador, dona accés a l'objecte al qual s'apunta

Recordatori de punters

Funcionament bàsic



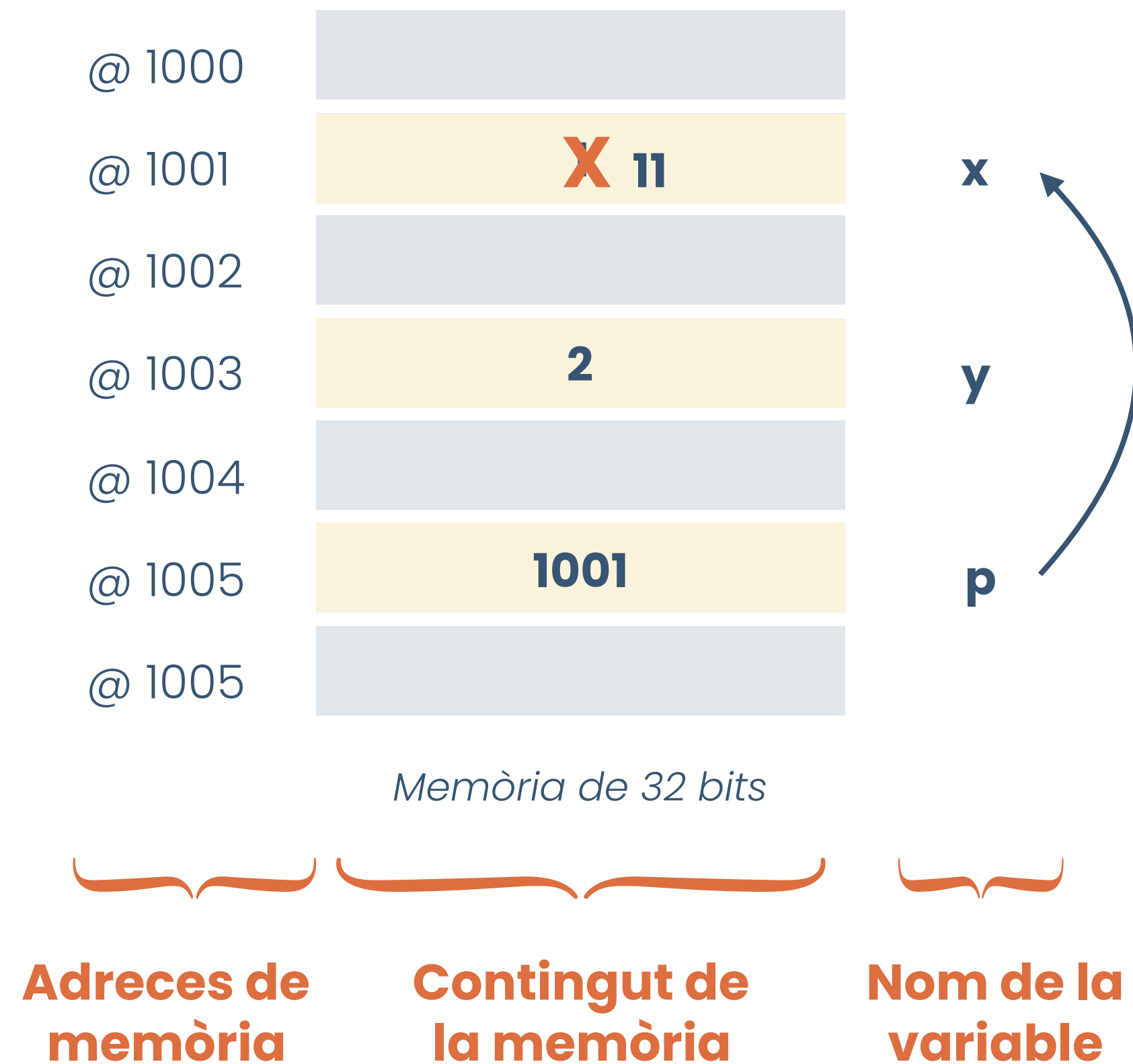
```
1 ...
2  int x;
3  int y;
4  int *p;
5
6  x = 1;
7  y = 2;
8  p = &x;
9  *p = *p + 10;
10 ...
```

Operador "&": et dóna l'adreça de memòria d'un objecte

Operador "*": aplicat a un apuntador, dona accés a l'objecte al qual s'apunta

Recordatori de punters

Funcionament bàsic



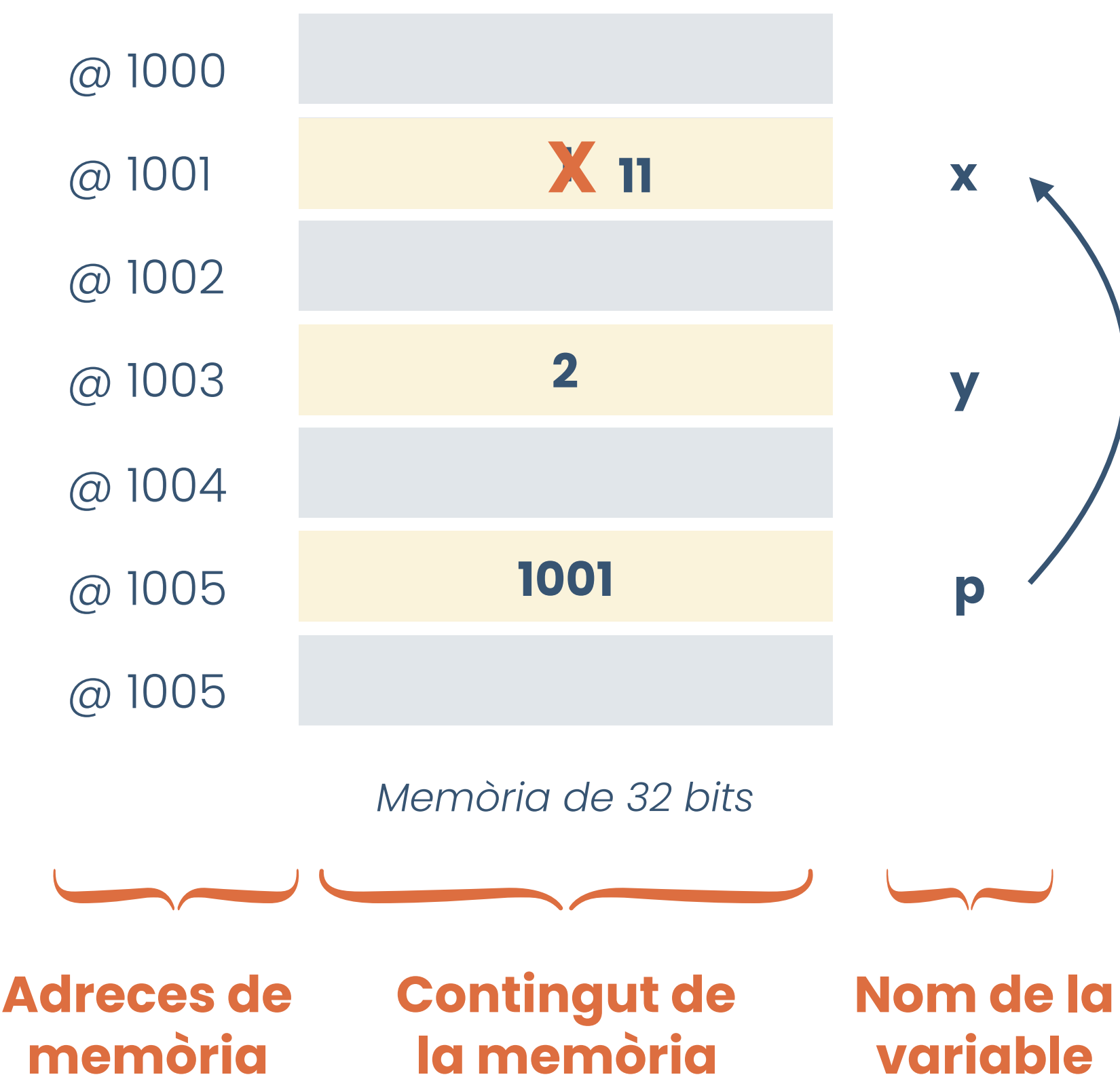
```
1 ...  
2     int x;  
3     int y;  
4     int *p;  
5  
6     x = 1;  
7     y = 2;  
8     p = &x;  
9     *p = *p + 10;  
10 ...
```

Operador "&": et dóna l'adreça de memòria d'un objecte

Operador "*": aplicat a un apuntador, dona accés a l'objecte al qual s'apunta

Recordatori de punters

Funcionament bàsic





OJO CUIDAO!

Què passa si
faig `*p=*p+10`
abans de fer
`p=&x`?

```
1 ...
2  int x;
3  int y;
4  int *p;
5
6  x = 1;
7  y = 2;
8  p = &x;
9  *p = *p + 10;
10 ...
```

Operador "&": et dóna l'adreça de memòria d'un objecte
Operador "*": aplicat a un apuntador, dona accés a l'objecte al qual s'apunta

Variables vs taules

Distinció important



```
1 #include <stdio.h>
2 int main(){
3     int x = 200;
4     int v[3] = {1,2,3};
5     int *p;
6
7
8
9     Exemple amb variables simples
10
11
12
13
14
15     Exemple amb taules
16
17
18
19 }
```

Variables vs taules

Distinció important



```
1 #include <stdio.h>
2 int main(){
3     int x = 200;
4     int v[3] = {1,2,3};
5     int *p;
6
7     p = &x;
8     printf("x = %u\n", x);
9     printf("&x = %u\n", &x);
10    printf("p = %u\n", p);
11    printf("*p = %d\n", *p);
12
13
14
15    Exemple amb taules
16
17
18
19 }
```

Variables vs taules

Distinció important

```
x    = 200
&x   = 3814328964
p    = 3814328964
*p   = 200
```

- El nom d'una variable simple fa referència al seu valor

```
1 #include <stdio.h>
2 int main(){
3     int x = 200;
4     int v[3] = {1,2,3};
5     int *p;
6
7     p = &x;
8     printf("x = %u\n", x);
9     printf("&x = %u\n", &x);
10    printf("p = %u\n", p);
11    printf("*p = %d\n", *p);
12
13
14
15    Exemple amb taules
16
17
18
19 }
```


Variables vs taules

Distinció important

```
x    = 200
&x   = 3814328964
p    = 3814328964
*p   = 200
```

- El nom d'una variable simple fa referència al seu valor

```
1 #include <stdio.h>
2 int main(){
3     int x = 200;
4     int v[3] = {1,2,3};
5     int *p;
6
7     p = &x;
8     printf("x = %u\n", x);
9     printf("&x = %u\n", &x);
10    printf("p = %u\n", p);
11    printf("*p = %d\n", *p);
12
13    p = &v;
14    printf("v = %u\n", v);
15    printf("&v = %u\n", &v);
16    printf("v[0] = %u\n", v[0]);
17    printf("p = %u\n", p);
18    printf("*p = %d\n", *p);
19 }
```

Variables vs taules

Distinció important

```
x    = 200
&x   = 3814328964
p    = 3814328964
*p   = 200
```

```
v      = 3814328972
&v     = 3814328972
v[0]   = 1
p      = 3814328972
*p     = 1
```

- El nom d'una variable simple fa referència al seu valor
- El nom d'una variable de tipus taula fa referència a la seva adreça. (Adreça on comença la taula).

```
1 #include <stdio.h>
2 int main(){
3     int x = 200;
4     int v[3] = {1,2,3};
5     int *p;
6
7     p = &x;
8     printf("x = %u\n", x);
9     printf("&x = %u\n", &x);
10    printf("p = %u\n", p);
11    printf("*p = %d\n", *p);
12
13    p = &v;
14    printf("v = %u\n", v);
15    printf("&v = %u\n", &v);
16    printf("v[0] = %u\n", v[0]);
17    printf("p = %u\n", p);
18    printf("*p = %d\n", *p);
19 }
```

Pas de paràmetres per valor

En C, el pas de paràmetre sempre es fa per valor. És a dir, sempre li passem una còpia d'un valor.

Com afecta això a les ...

Variables simples



```
1 #include <stdio.h>
2 void incrementa_num(int n, int inc){
3     n = n + inc;
4 }
5
6 int main(){
7     int x = 200;
8     incrementa_num(x, 10);
9     printf("Num = %d\n", x);
10 }
```


Pas de paràmetres per valor

En C, el pas de paràmetre sempre es fa per valor. És a dir, sempre li passem una còpia d'un valor.

Com afecta això a les ...

Variables simples

```
1 #include <stdio.h>
2 void incrementa_num(int n, int inc){
3     n = n + inc;
4 }
5
6 int main(){
7     int x = 200;
8     incrementa_num(x, 10);
9     printf("Num = %d\n", x);
10 }
```

Num = 200



Pas de paràmetres per valor

En C, el pas de paràmetre sempre es fa per valor. És a dir, sempre li passem una còpia d'un valor.

Com afecta això a les ...

Variables "taula"

```
1 #include <stdio.h>
2 #define MIDA 3
3
4 int main(){
5     int v[MIDA] = {1,2,3};
6     incrementa_taula(v, MIDA, 10);
7     print_taula(v, MIDA);
8 }
```

```
1 void incrementa_taula (int v[], int mida, int inc){
2     for(int i=0; i<mida; i++){
3         v[i] = v[i] + inc;
4     }
5 }
6
7 void print_taula (int v[], int mida){
8     for(int i=0; i<mida; i++){
9         printf("%d ", v[i]);
10    }
11    printf("\n");
12 }
```

Pas de paràmetres per valor

En C, el pas de paràmetre sempre es fa per valor. És a dir, sempre li passem una còpia d'un valor.

Com afecta això a les ...

Variables "taula"

```
1 #include <stdio.h>
2 #define MIDA 3
3
4 int main(){
5     int v[MIDA] = {1,2,3};
6     incrementa_taula(v, MIDA, 10);
7     print_taula(v, MIDA);
8 }
```

```
1 void incrementa_taula (int v[], int mida, int inc){
2     for(int i=0; i<mida; i++){
3         v[i] = v[i] + inc;
4     }
5 }
6
7 void print_taula (int v[], int mida){
8     for(int i=0; i<mida; i++){
9         printf("%d ", v[i]);
10    }
11    printf("\n");
12 }
```

11 12 13



Pas de paràmetres per valor

En C, el pas de paràmetre sempre es fa per valor. És a dir, sempre li passem una còpia d'un valor.

Com afecta això a les ...

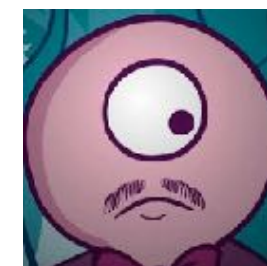
Variables "taula"

```
1 #include <stdio.h>
2 #define MIDA 3
3
4 int main(){
5     int v[MIDA] = {1,2,3};
6     incrementa_taula(v, MIDA, 10);
7     print_taula(v, MIDA);
8 }
```

11 12 13



```
1 void incrementa_taula (int v[], int mida, int inc){
2     for(int i=0; i<mida; i++){
3         v[i] = v[i] + inc;
4     }
5 }
6
7 void print_taula (int v[], int mida){
8     for(int i=0; i<mida; i++){
9         printf("%d ", v[i]);
10    }
11    printf("\n");
12 }
```



OJO CUIDAO!

Això vol dir que una funció que no pretén modificar una taula, també pot modificar-la! `print_taula` per exemple!

Simular el pas per referència

Com hem vist, les variables simples no poden ser modificades dins una funció.

Per poder-les modificar, hem de simular un pas per referència: passar-li l'adreça de memòria de la variable a modificar

Variables simples: pas per referència

```
1 #include <stdio.h>
2
3 void incrementa_num(int *n, int inc){
4     *n = *n + inc;
5 }
6
7 int main(){
8     int x = 200;
9     // Pas per referencia
10    incrementa_num(&x, 10);
11    printf("Num = %d\n", x);
12 }
```

Simular el pas per referència

Com hem vist, les variables simples no poden ser modificades dins una funció.

Per poder-les modificar, hem de simular un pas per referència: passar-li l'adreça de memòria de la variable a modificar

Variables simples: pas per referència

```
1 #include <stdio.h>
2
3 void incrementa_num(int *n, int inc){
4     *n = *n + inc;
5 }
6
7 int main(){
8     int x = 200;
9     // Pas per referencia
10    incrementa_num(&x, 10);
11    printf("Num = %d\n", x);
12 }
```

Necessitem usar "&" per passar l'adreça de memòria de la variable

Simular el pas per referència

Com hem vist, les variables simples no poden ser modificades dins una funció.

Per poder-les modificar, hem de simular un pas per referència: passar-li l'adreça de memòria de la variable a modificar

Variables simples: pas per referència

```
1 #include <stdio.h>
2
3 void incrementa_num(int *n, int inc){
4     *n = *n + inc;
5 }
6
7 int main(){
8     int x = 200;
9     // Pas per referencia
10    incrementa_num(&x, 10);
11    printf("Num = %d\n", x);
12 }
```

Ara la funció ja no rep el valor de la variable, sinó el valor de l'adreça de memòria de la variable

Necessitem usar "&" per passar l'adreça de memòria de la variable

Simular el pas per referència

Com hem vist, les variables simples no poden ser modificades dins una funció.

Per poder-les modificar, hem de simular un pas per referència: passar-li l'adreça de memòria de la variable a modificar

Variables simples: pas per referència

```
1 #include <stdio.h>
2
3 void incrementa_num(int *n, int inc){
4     *n = *n + inc;
5 }
6
7 int main(){
8     int x = 200;
9     // Pas per referencia
10    incrementa_num(&x, 10);
11    printf("Num = %d\n", x);
12 }
```

Ara la funció ja no rep el valor de la variable, sinó el valor de l'adreça de memòria de la variable

Com que hem rebut l'adreça de memòria, hem de fer servir l'operador "*" per accedir al valor

Necessitem usar "&" per passar l'adreça de memòria de la variable

Simular el pas per referència

Com hem vist, les variables simples no poden ser modificades dins una funció.

Per poder-les modificar, hem de simular un pas per referència: passar-li l'adreça de memòria de la variable a modificar

Variables simples: pas per referència

```
1 #include <stdio.h>
2
3 void incrementa_num(int *n, int inc){
4     *n = *n + inc;
5 }
6
7 int main(){
8     int x = 200;
9     // Pas per referencia
10    incrementa_num(&x, 10);
11    printf("Num = %d\n", x);
12 }
```

Ara la funció ja no rep el valor de la variable, sinó el valor de l'adreça de memòria de la variable

Com que hem rebut l'adreça de memòria, hem de fer servir l'operador "*" per accedir al valor

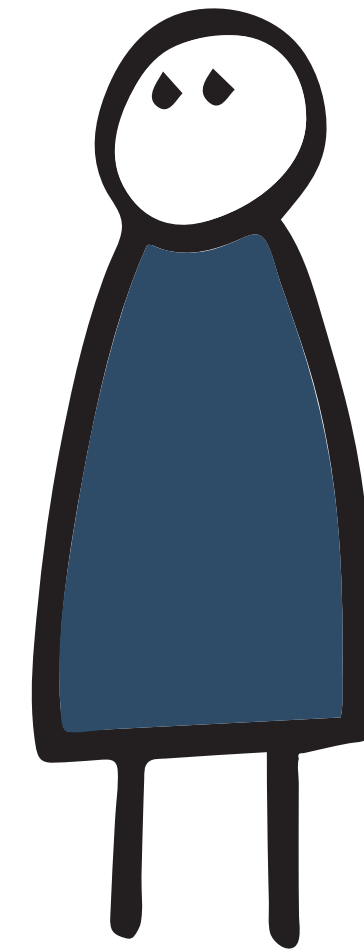
Necessitem usar "&" per passar l'adreça de memòria de la variable

Num = 210



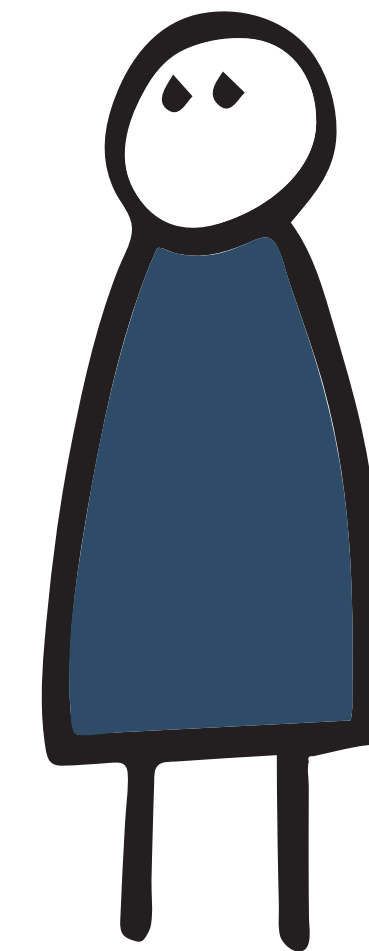
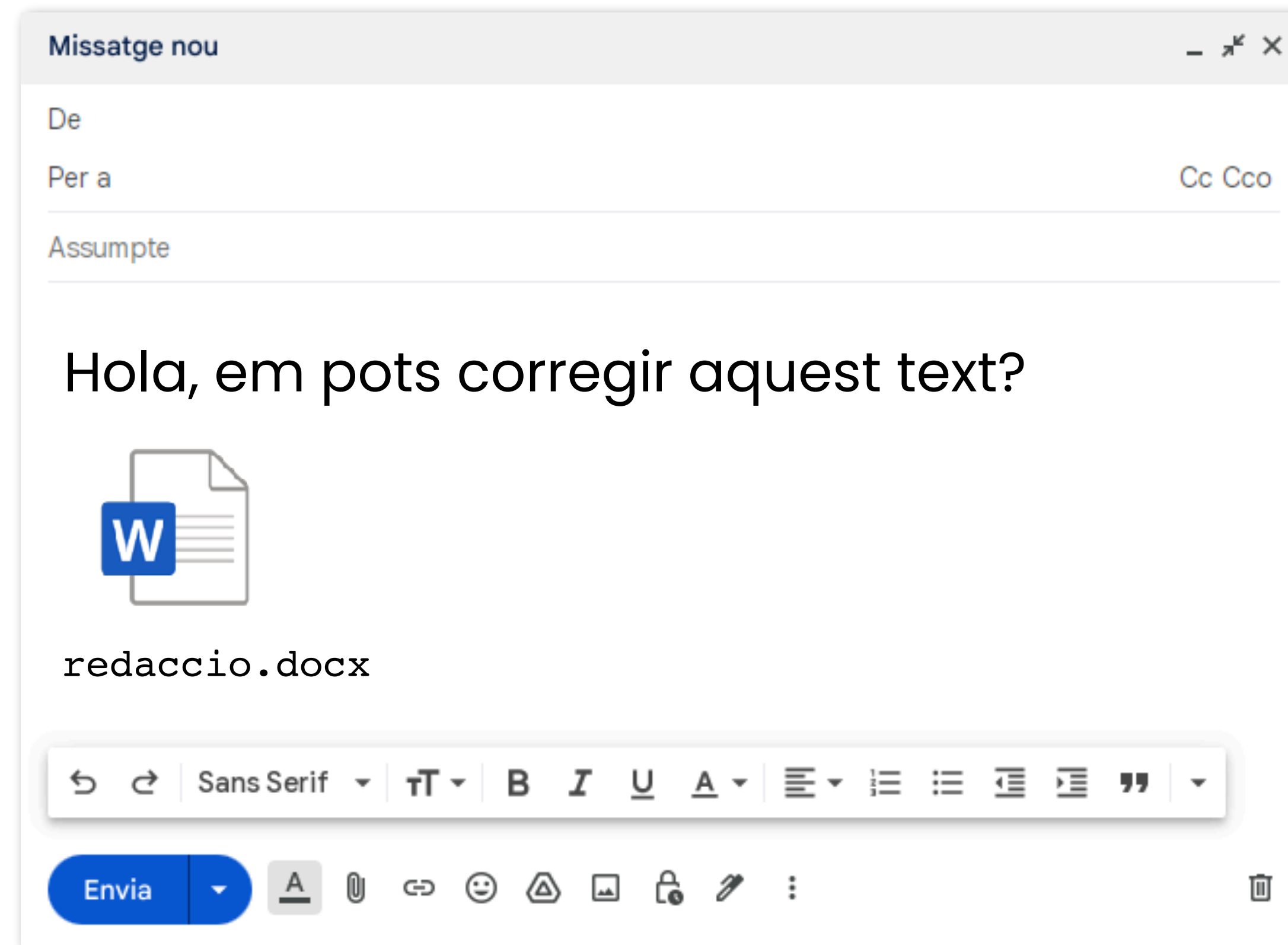
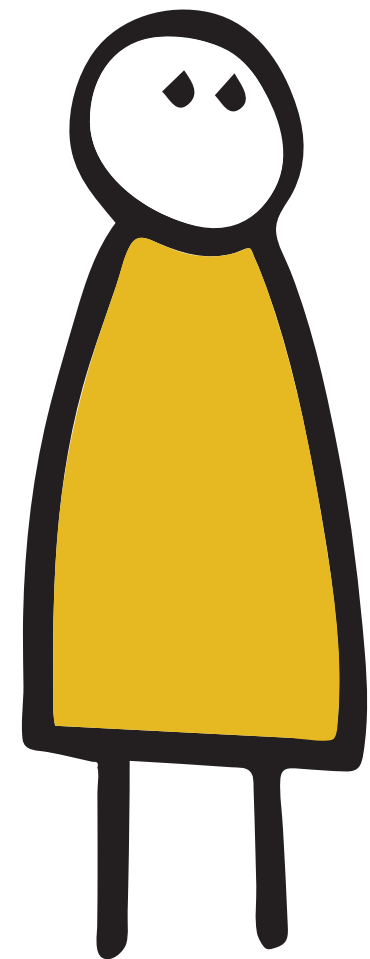
Pas per valor vs pas per referència a la vida real

Pas per valor



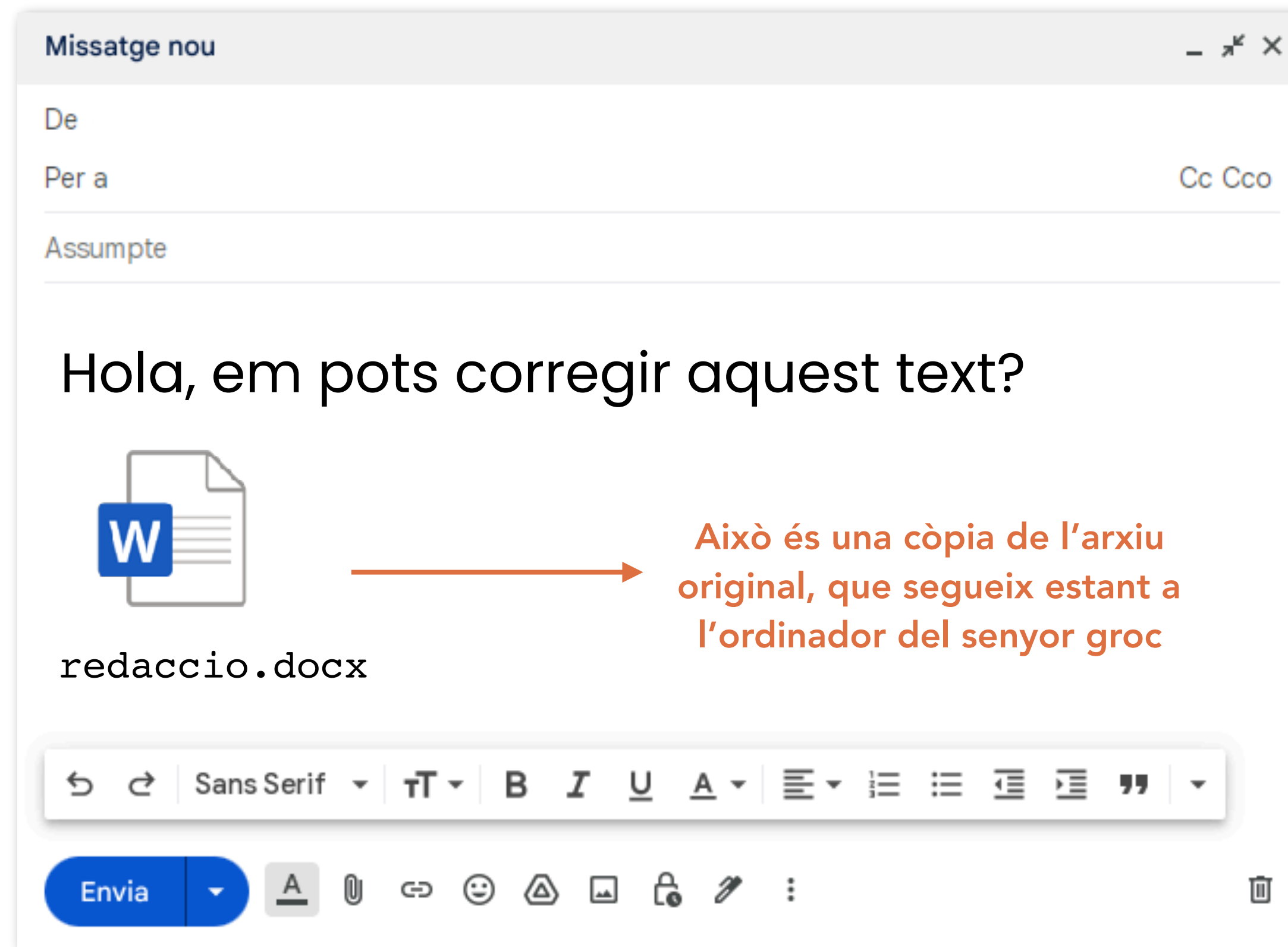
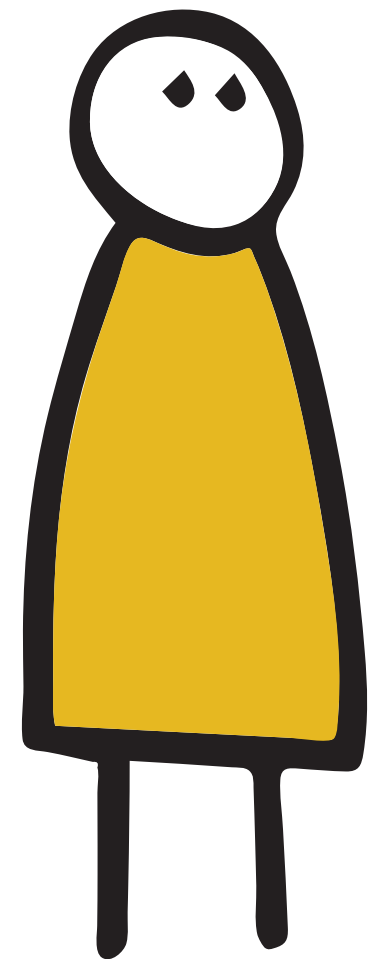
Pas per valor vs pas per referència a la vida real

Pas per valor

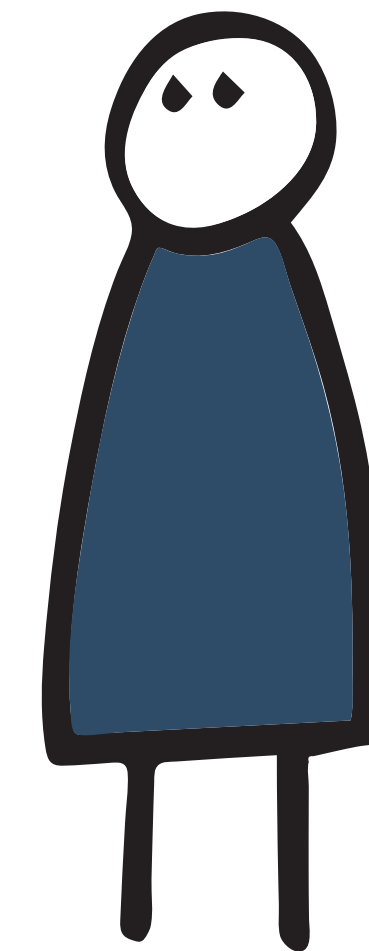


Pas per valor vs pas per referència a la vida real

Pas per valor

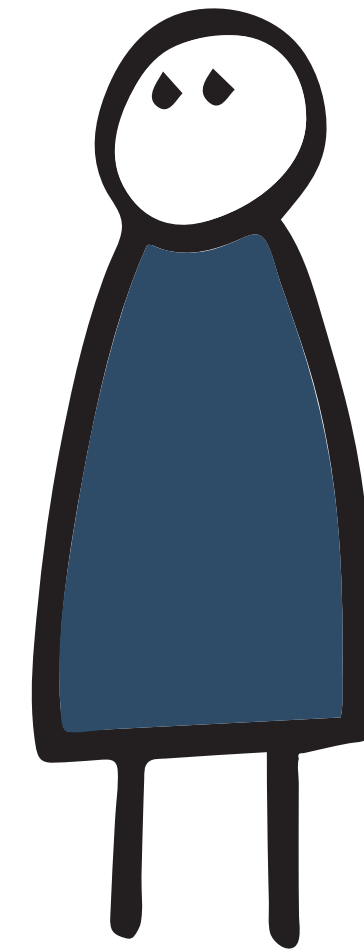


Hauré de modificar el document i tornar-li a enviar (retornar-li) si vull que vegi els canvis



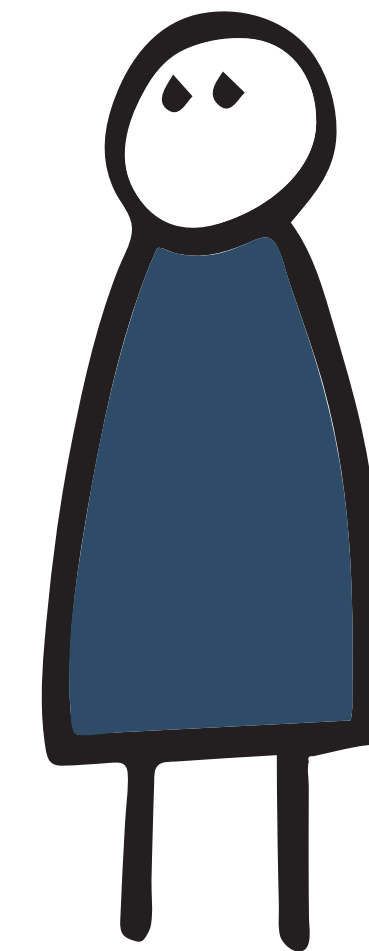
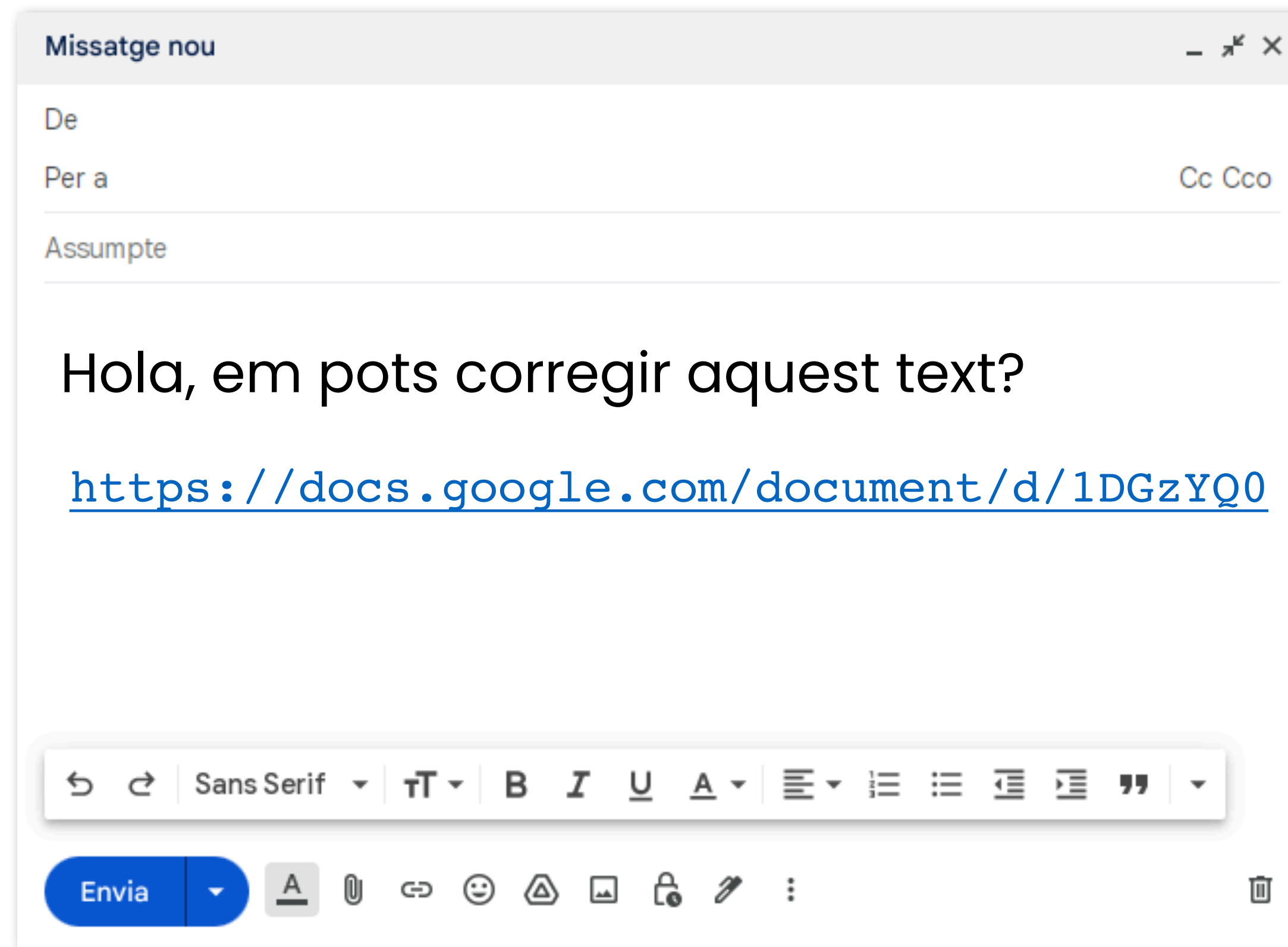
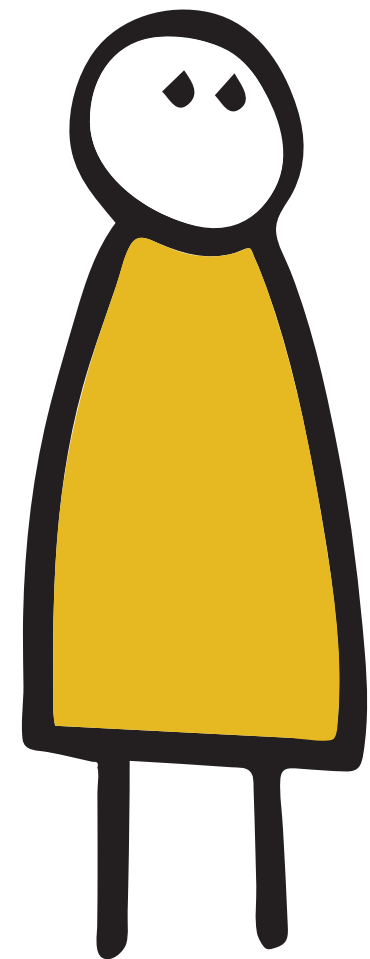
Pas per valor vs pas per referència a la vida real

Pas per referència



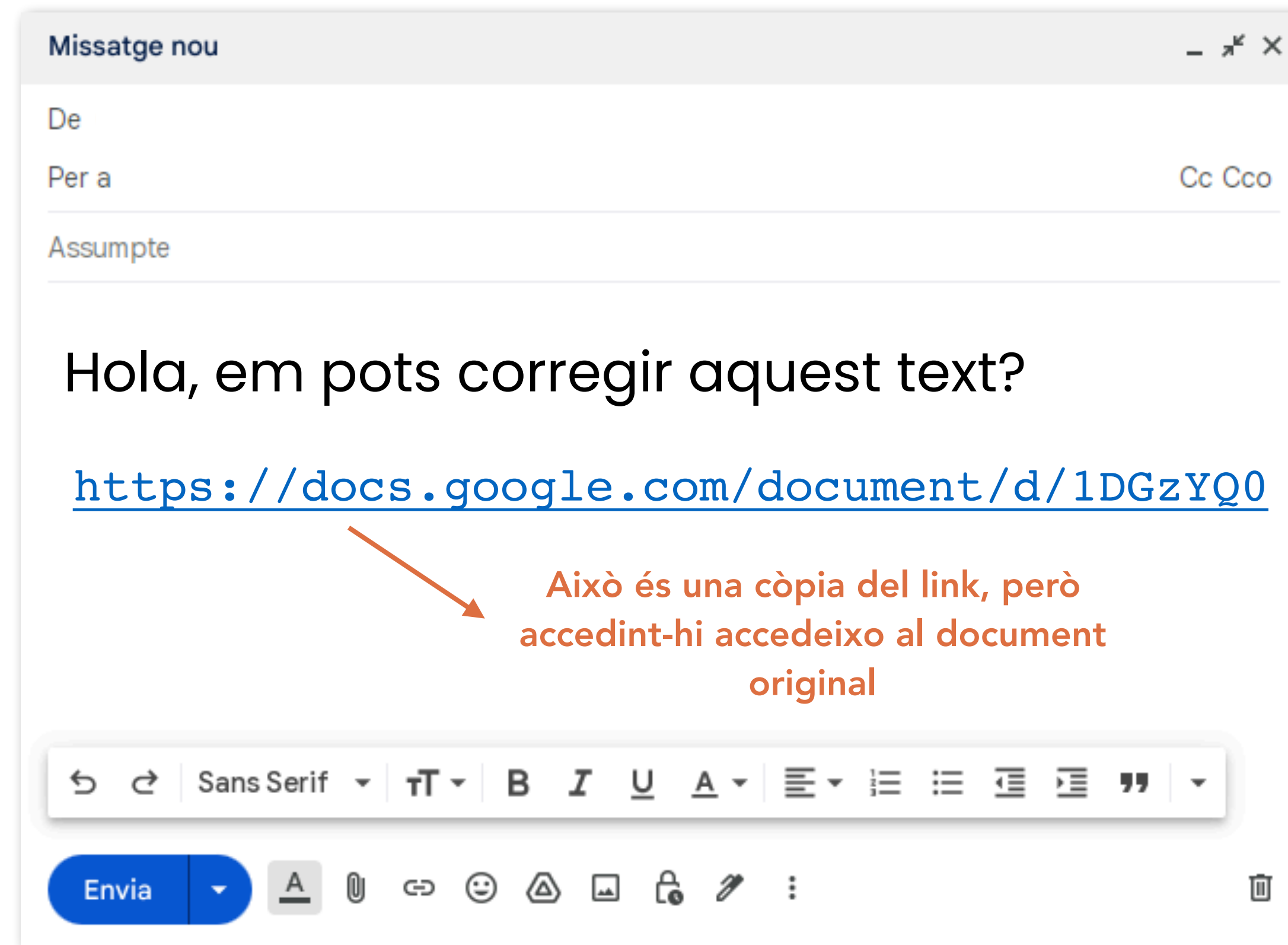
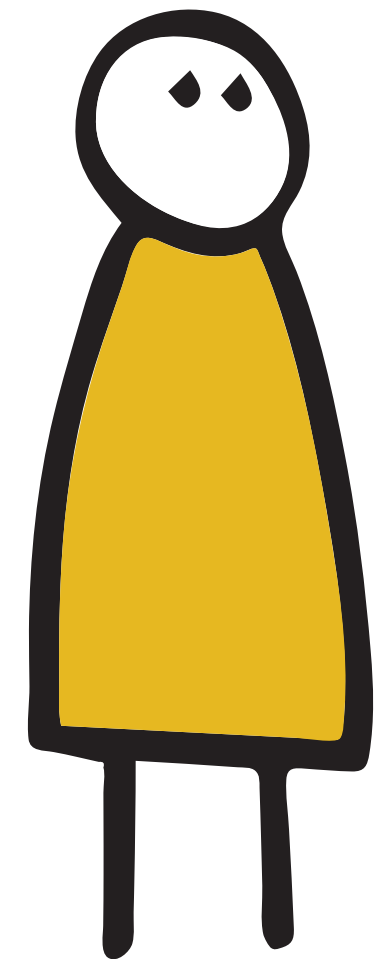
Pas per valor vs pas per referència a la vida real

Pas per referència

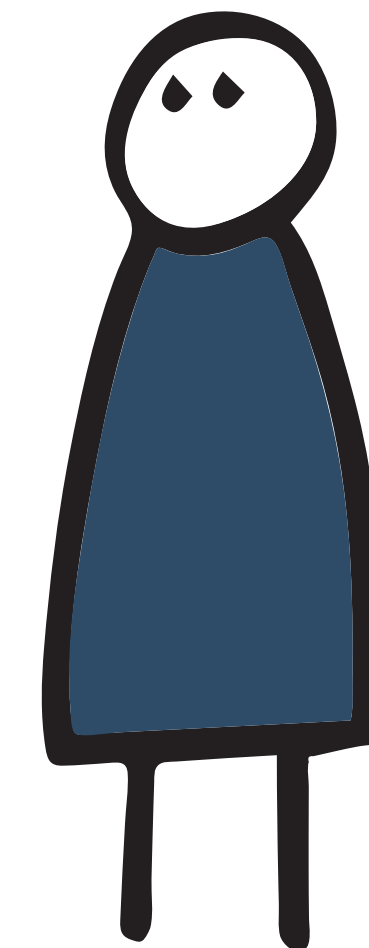


Pas per valor vs pas per referència a la vida real

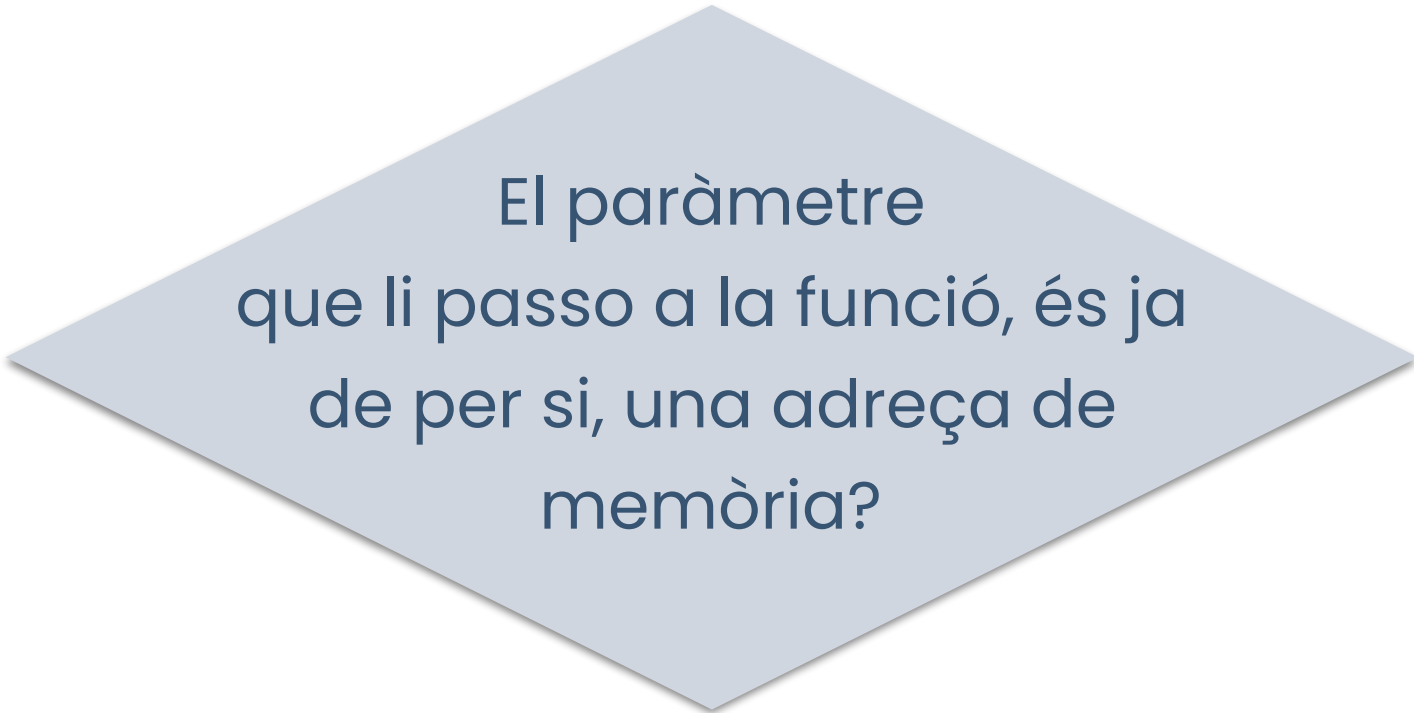
Pas per referència



No cal que li retorni, només cal que accedeixi al fitxer original i el canviï

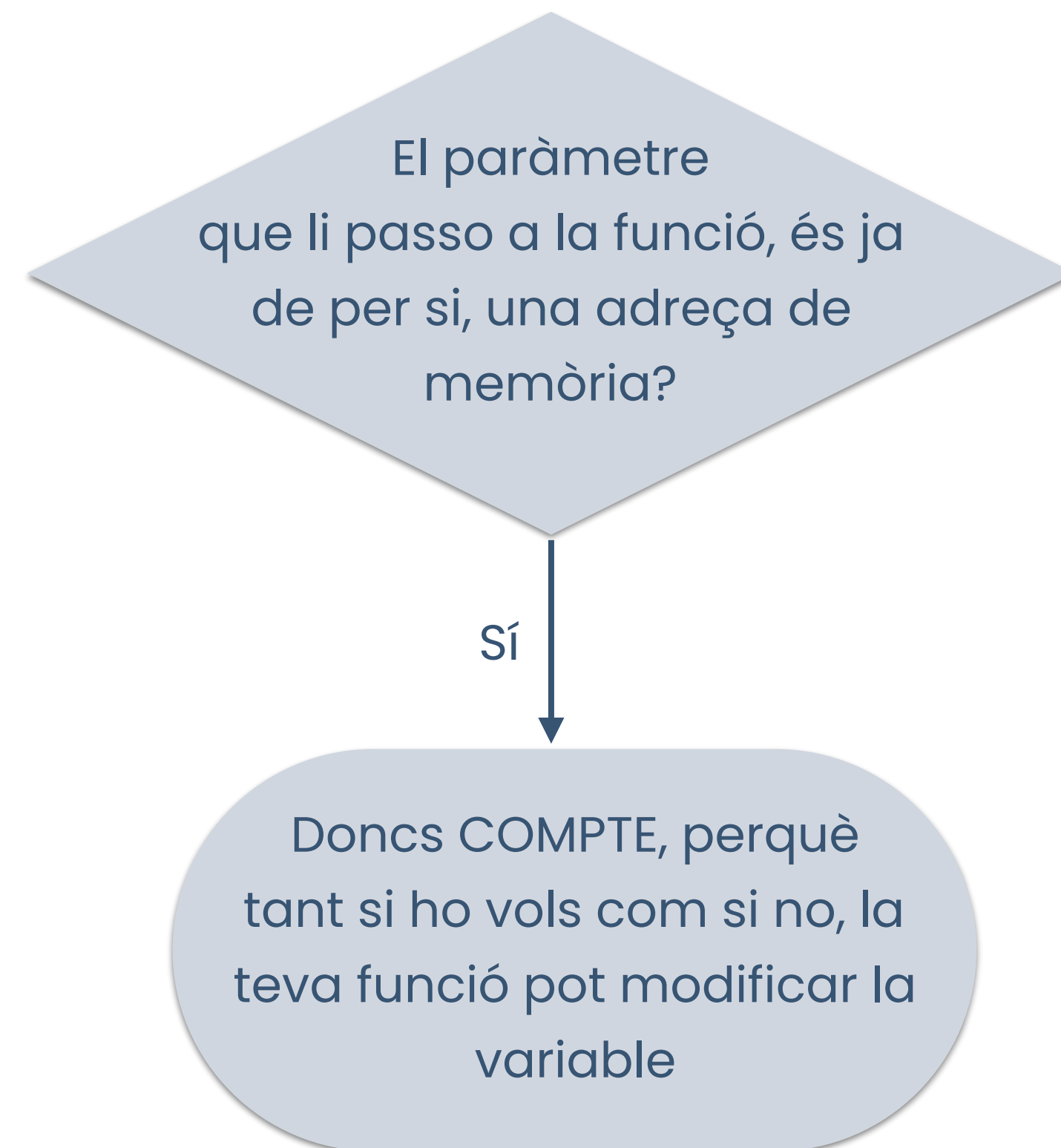


Quan necessitem punters en una funció?

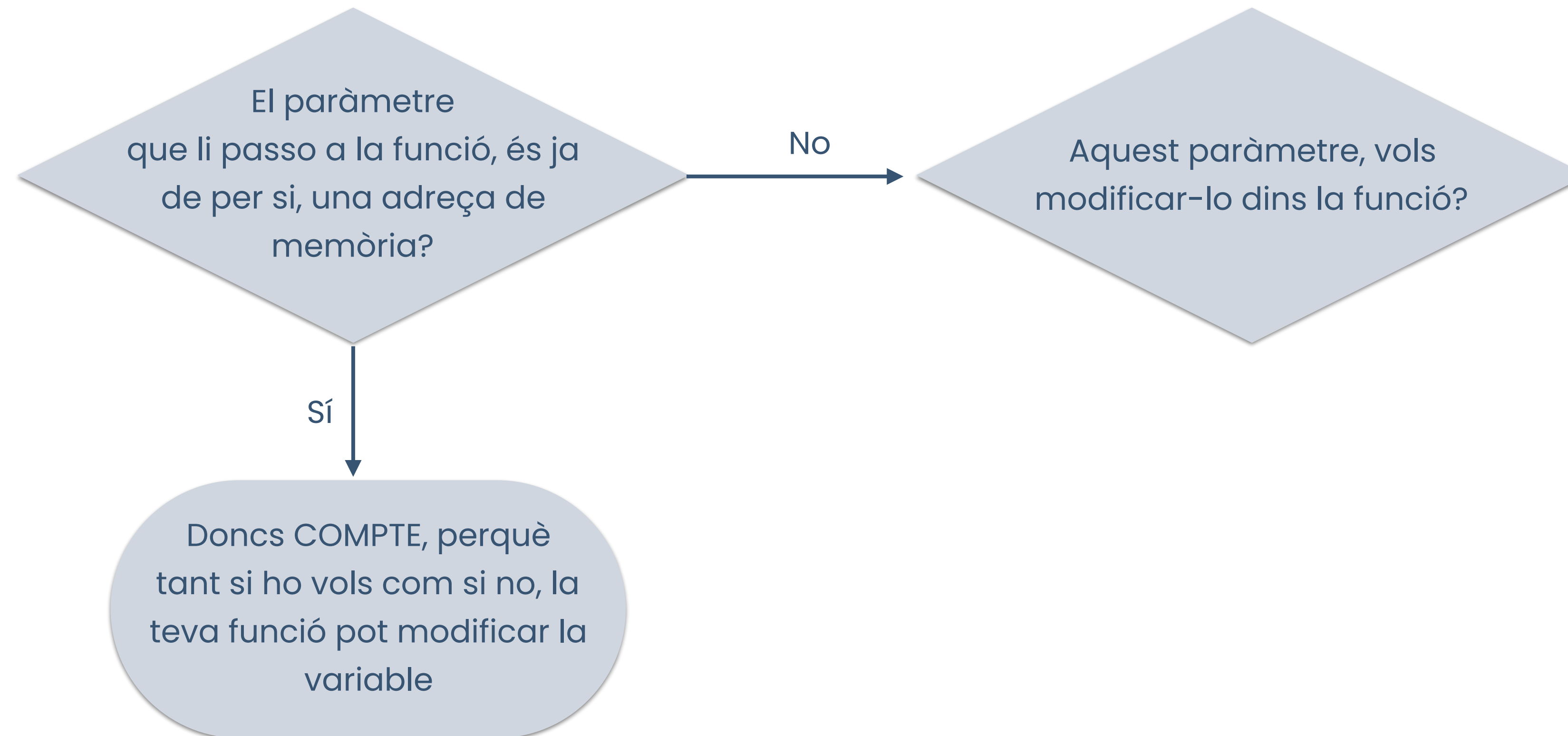


El paràmetre
que li passo a la funció, és ja
de per si, una adreça de
memòria?

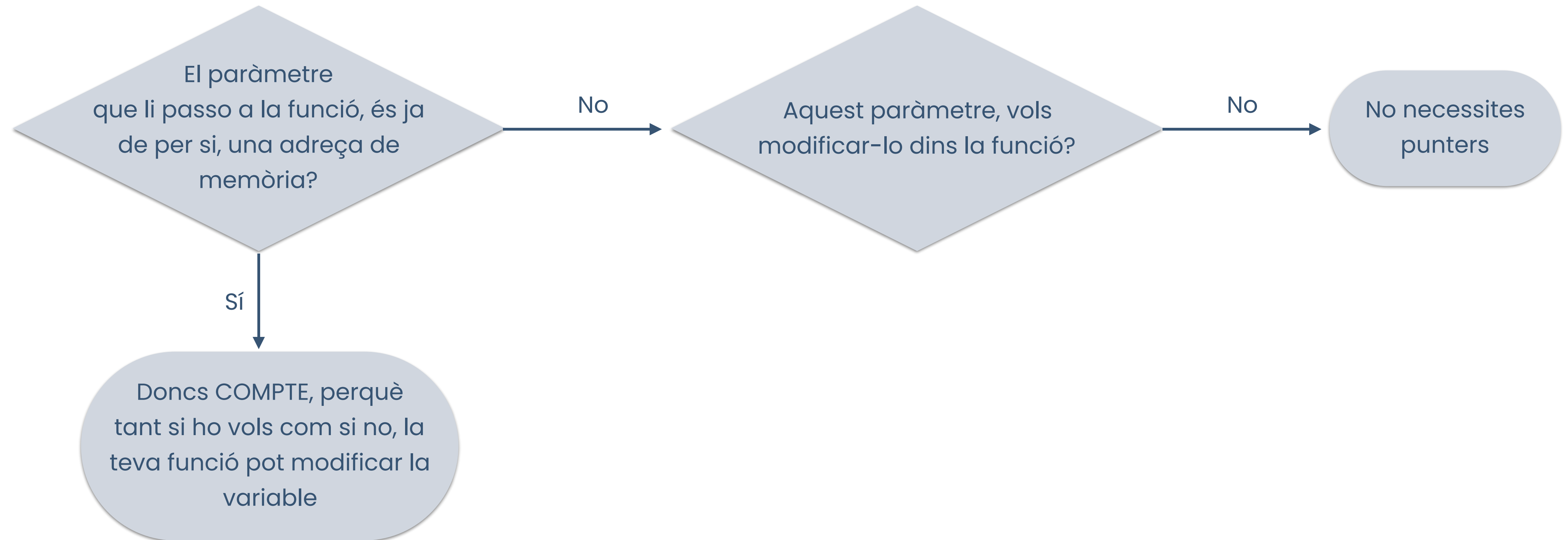
Quan necessitem punters en una funció?



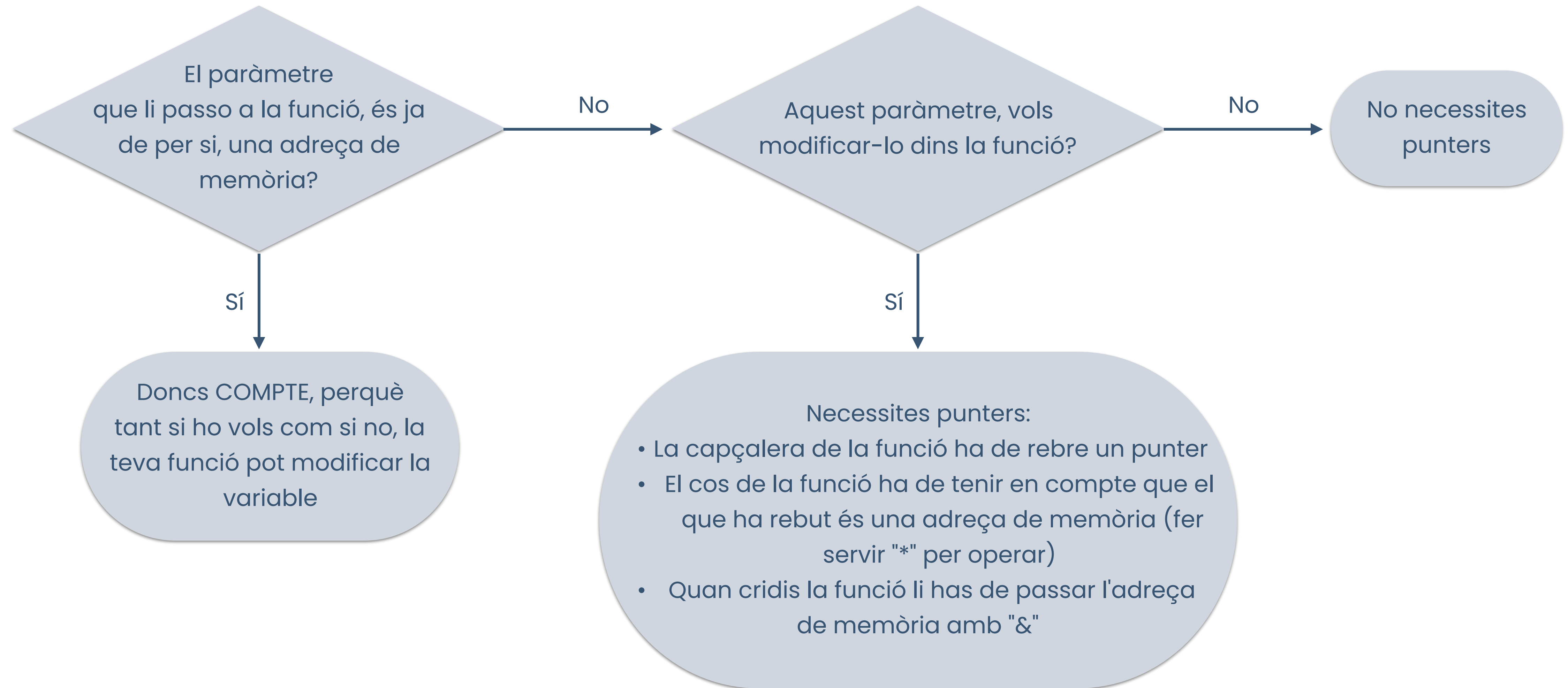
Quan necessitem punters en una funció?



Quan necessitem punters en una funció?



Quan necessitem punters en una funció?



REPASSEM...

MEMÒRIA DINÀMICA

Taules en memòria dinàmica

- Ja hem vist que el nom de qualsevol taula (estàtica o dinàmica) fa referència a una adreça de memòria.
- Mireu de quin tipus és la variable **v**

Taules en memòria dinàmica

- Ja hem vist que el nom de qualsevol taula (estàtica o dinàmica) fa referència a una adreça de memòria.
- Mireu de quin tipus és la variable **v**

```
1 #include <stdio.h>
2
3 int main(){
4     int v[5];
5     printf("v=%d\n",v);
6 }
```

```
$ gcc test.c -o test
test.c:9:18: warning: format specifies type 'int' but the argument has type 'int *' [-Wformat]
    printf("v=%d\n",v);
```

Taules en memòria dinàmica

- Ja hem vist que el nom de qualsevol taula (estàtica o dinàmica) fa referència a una adreça de memòria.

Taules en memòria dinàmica

- Ja hem vist que el nom de qualsevol taula (estàtica o dinàmica) fa referència a una adreça de memòria.
- Qualsevol taula és un punter al tipus de la taula
- Per això puc especificar-les com a paràmetre d'aquestes dues maneres i les dues funcionen tant per taules estàtiques com dinàmiques

Taules en memòria dinàmica

- Ja hem vist que el nom de qualsevol taula (estàtica o dinàmica) fa referència a una adreça de memòria.
- Qualsevol taula és un punter al tipus de la taula
- Per això puc especificar-les com a paràmetre d'aquestes dues maneres i les dues funcionen tant per taules estàtiques com dinàmiques

```
1 #include <stdio.h>
2 #define MIDA 5
3
4 void imprimeix_taula (int v[], int mida){
5     for(int i = 0; i < mida; i++){
6         printf("%d ", v[i]);
7     }
8     printf("\n");
9 }
10
11 int main(){
12     int v[MIDA] = {1,2,3,4,5};
13     imprimeix_taula(v, MIDA);
14 }
```

```
1 #include <stdio.h>
2 #define MIDA 5
3
4 void imprimeix_taula (int *v, int mida){
5     for(int i = 0; i < mida; i++){
6         printf("%d ", v[i]);
7     }
8     printf("\n");
9 }
10
11 int main(){
12     int v[MIDA] = {1,2,3,4,5};
13     imprimeix_taula(v, MIDA);
14 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

```
1 #include <stdio.h>
2
3
4
5
6
7
8
9
10 int main(){
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**

```
1 #include <stdio.h>
2
3
4
5
6
7
8
9
10 int main(){
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```


Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```


Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```


Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12     double * v;
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent

```
void *malloc(size_t size)
```

```
void *calloc(size_t nitems, size_t size)
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12     double * v;
13
14
15
16
17
18
19
20
21
22
23
24
25
26 }
```


Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent

```
void *malloc(size_t size)
```

```
void *calloc(size_t nitems, size_t size)
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12     double * v;
13
14
15     v = (double *) malloc(n * sizeof(double));
16
17
18
19
20
21
22
23
24
25
26 }
```


Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12     double * v;
13
14
15     v = (double *) malloc(n * sizeof(double));
16
17
18
19
20
21
22
23
24
25
26 }
```


Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent
- Comprovarem que l'assignació sigui correcta

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12     double * v;
13
14
15     v = (double *) malloc(n * sizeof(double));
16
17
18
19
20
21
22
23
24
25
26 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent
- Comprovarem que l'assignació sigui correcta

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12     double * v;
13
14
15     v = (double *) malloc(n * sizeof(double));
16     if (v == NULL){
17         printf("ERROR memoria\n");
18         return -1;
19     }
20
21
22
23
24
25
26 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent
- Comprovarem que l'assignació sigui correcta
- Farem servir el vector amb normalitat

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5
6
7
8
9
10 int main(){
11     int n = 10;
12     double * v;
13
14
15     v = (double *) malloc(n * sizeof(double));
16     if (v == NULL){
17         printf("ERROR memoria\n");
18         return -1;
19     }
20
21
22
23
24
25
26 }
```


Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent
- Comprovarem que l'assignació sigui correcta
- Farem servir el vector amb normalitat

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void print_vd (double v[], int mida){
5     for(int i=0; i<mida; i++){
6         printf("%.5f\n", v[i]);
7     }
8 }
9
10 int main(){
11     int n = 10;
12     double * v;
13     srand(123);
14
15     v = (double *) malloc(n * sizeof(double));
16     if (v == NULL){
17         printf("ERROR memoria\n");
18         return -1;
19     }
20
21     for(int i=0; i<n; i++){
22         v[i] = (double) rand() / RAND_MAX;
23     }
24     print_vd(v, n);
25
26 }
```

Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent
- Comprovarem que l'assignació sigui correcta
- Farem servir el vector amb normalitat
- Quan haguem acabat, alliberarem la memòria amb **free**

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void print_vd (double v[], int mida){
5     for(int i=0; i<mida; i++){
6         printf("%.5f\n", v[i]);
7     }
8 }
9
10 int main(){
11     int n = 10;
12     double * v;
13     srand(123);
14
15     v = (double *) malloc(n * sizeof(double));
16     if (v == NULL){
17         printf("ERROR memoria\n");
18         return -1;
19     }
20
21     for(int i=0; i<n; i++){
22         v[i] = (double) rand() / RAND_MAX;
23     }
24     print_vd(v, n);
25
26 }
```


Taules en memòria dinàmica

Com declarar taules dinàmiques 1D: **RECEPTA**

- Necessitem tenir inclosa la llibreria **<stdlib.h>**, que és on hi ha les funcions **malloc**, **calloc**, **realloc** i **free**
- Necessitem saber què guardarem: int? char? double? float? un tipus d'usuari (registre)?...
- Necessitem saber quants elements guardarem.
- Declararem un punter al tipus
- Cridarem la funció **malloc** (o **calloc**) especificant la mida de cada element i el nombre d'elements i fem el càsting corresponent
- Comprovarem que l'assignació sigui correcta
- Farem servir el vector amb normalitat
- Quan haguem acabat, alliberarem la memòria amb **free**

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void print_vd (double v[], int mida){
5     for(int i=0; i<mida; i++){
6         printf("%.5f\n", v[i]);
7     }
8 }
9
10 int main(){
11     int n = 10;
12     double * v;
13     srand(123);
14
15     v = (double *) malloc(n * sizeof(double));
16     if (v == NULL){
17         printf("ERROR memoria\n");
18         return -1;
19     }
20
21     for(int i=0; i<n; i++){
22         v[i] = (double) rand() / RAND_MAX;
23     }
24     print_vd(v, n);
25     free(v);
26 }
```

Taules en memòria dinàmica

Com declarar taules 2D: recordatori

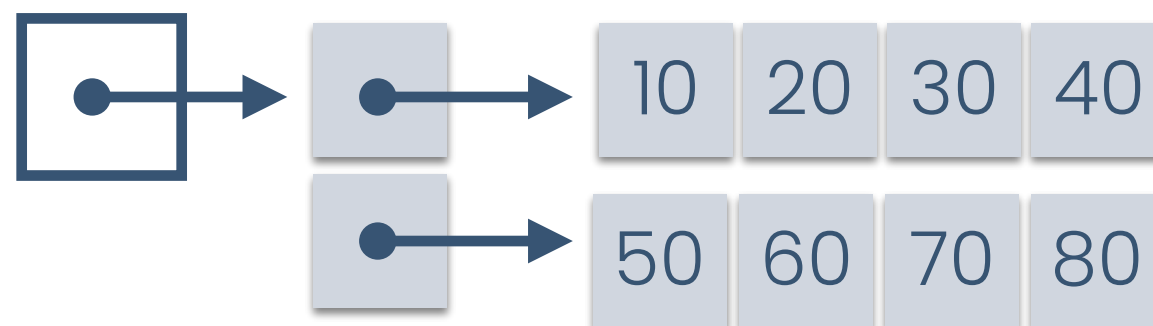
10	20	30	40
50	60	70	80

FIL = 2
COL = 4



Punter a
punter a enter

v



Vector **dinàmic** de
punters a enter

```
1 ...
2     /* Punter a punter a enter */
3     int ** v;
4     // Obrir fitxer i comprovacions (...)
5     // Llegir dues primeres línies i guardar a num_fil i num_col
6     fscanf(f_in, "%s%d", s, &num_fil);
7     fscanf(f_in, "%s%d", s, &num_col);
8
9     /* Alocatar memòria per la taula de punters a enter*/
10    v = (int**) malloc (num_fil * sizeof(int*));
11    // Comprovació memòria (...)
12    /* Alocatar memòria per les taules d'enters*/
13    for(int i = 0; i < num_fil; i++){
14        v[i] = (int*) malloc (num_col * sizeof(int));
15        // Comprovació memòria (...)
16    }
17
18    /* Accés normal: */
19    for(int i=0; i < num_fil; i++){
20        for(int j=0; j < num_col; j++){
21            fscanf(f_in, "%d", &num);
22            v[i][j] = num;
23        }
24    }
25 ...
```

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors $\{1,2,3,4,5\}$ i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

Com fer-ho?

- Si no necessitem mantenir els valors originals, podem fer una acció que modifiqui el vector original:

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

Com fer-ho?

- Si no necessitem mantenir els valors originals, podem fer una acció que modifiqui el vector original:

```
1 #include <stdio.h>
2 #define MIDA 5
3
4 int main(){
5     int v[MIDA] = {1,2,3,4,5};
6
7     print_taula(v, MIDA);
8     calcula_quadrats(v, MIDA);
9     print_taula(v, MIDA);
10
11 }
```

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

Com fer-ho?

- Si no necessitem mantenir els valors originals, podem fer una acció que modifiqui el vector original:

```
1 #include <stdio.h>
2 #define MIDA 5
3
4 int main(){
5     int v[MIDA] = {1,2,3,4,5};
6
7     print_taula(v, MIDA);
8     calcula_quadrats(v, MIDA);
9     print_taula(v, MIDA);
10
11 }
```

```
1 void calcula_quadrats(int v[], int mida){
2     for(int i=0; i<mida; i++){
3         v[i] = v[i]*v[i];
4     }
5 }
```

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

Com fer-ho?

- Si no necessitem mantenir els valors originals, podem fer una acció que modifiqui el vector original:

```
1 #include <stdio.h>
2 #define MIDA 5
3
4 int main(){
5     int v[MIDA] = {1,2,3,4,5};
6
7     print_taula(v, MIDA);
8     calcula_quadrats(v, MIDA);
9     print_taula(v, MIDA);
10
11 }
```

```
1 void calcula_quadrats(int v[], int mida){
2     for(int i=0; i<mida; i++){
3         v[i] = v[i]*v[i];
4     }
5 }
```

```
1 void print_taula (int v[], int mida){
2     for(int i=0; i<mida; i++){
3         printf("%d ", v[i]);
4     }
5     printf("\n");
6 }
```


Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

Com fer-ho?

- Si no necessitem mantenir els valors originals, podem fer una acció que modifiqui el vector original:

```
1 #include <stdio.h>
2 #define MIDA 5
3
4 int main(){
5     int v[MIDA] = {1,2,3,4,5};
6
7     print_taula(v, MIDA);
8     calcula_quadrats(v, MIDA);
9     print_taula(v, MIDA);
10
11 }
```

1	2	3	4	5
1	4	9	16	25

```
1 void calcula_quadrats(int v[], int mida){
2     for(int i=0; i<mida; i++){
3         v[i] = v[i]*v[i];
4     }
5 }
```

```
1 void print_taula (int v[], int mida){
2     for(int i=0; i<mida; i++){
3         printf("%d ", v[i]);
4     }
5     printf("\n");
6 }
```

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors $\{1,2,3,4,5\}$ i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

Com fer-ho?

- Si necessitem mantenir els valors originals, hem de crear un nou vector amb els continguts.
- **Opció 1:** Fem una funció que rebi per paràmetre el vector original i retorni el vector modificat

`v2 = calcular_quadrats(v);`

- **Opció 2:** Fem una acció que rebi per paràmetre el vector original, un vector buit, i en modifiqui aquest segon.

`calcular_quadrats(v, v2);`

- Com fer cada opció correctament?

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors $\{1,2,3,4,5\}$ i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 1:** Fem una funció que rebi per paràmetre el vector original i retorni el vector modificat

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 1:** Fem una funció que rebi per paràmetre el vector original i retorni el vector modificat

```
1 int main(){
2
3     int v[MIDA] = {1,2,3,4,5};
4     // Opcio 1
5     int * v2;
6     print_taula(v, MIDA);
7     v2 = calcular_quadrats(v, MIDA);
8     print_taula(v2, MIDA);
9 }
```


Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 1:** Fem una funció que rebi per paràmetre el vector original i retorni el vector modificat

```
1 int main(){
2
3     int v[MIDA] = {1,2,3,4,5};
4     // Opcio 1
5     int * v2;
6     print_taula(v, MIDA);
7     v2 = calcular_quadrats(v, MIDA);
8     print_taula(v2, MIDA);
9 }
```

PROGRAMA PRINCIPAL

```
1 int * calcular_quadrats(int * v, int mida){
2     // Reservem la memòria aquí
3     int * v2;
4     v2 = (int *) malloc (mida * sizeof(int));
5     if (v2 == NULL){
6         printf("Error memòria.\n");
7         return -1;
8     }
9     for(int i=0; i < mida; i++){
10         v2[i] = v[i]*v[i];
11     }
12     return v2;
13 }
```

FUNCIÓ CALCULAR_QUADRATS

Procediments que modifiquen taules: acció o funció?

Imaginem que
quadrats d'a

- **Opció 1:** Fe

warning: incompatible integer to pointer conversion returning 'int' from a function
with result

```
type 'int *' [-Wint-conversion]  
return -1;  
^~
```

1 warning generated.

amb els



```
1 int main(){  
2  
3     int v[MIDA] = {1,2,3,4,5};  
4     // Opcio 1  
5     int * v2;  
6     print_taula(v, MIDA);  
7     v2 = calcular_quadrats(v, MIDA);  
8     print_taula(v2, MIDA);  
9 }
```

PROGRAMA PRINCIPAL



```
1 int * calcular_quadrats(int * v, int mida){  
2     // Reservem la memòria aquí  
3     int * v2;  
4     v2 = (int *) malloc (mida * sizeof(int));  
5     if (v2 == NULL){  
6         printf("Error memòria.\n");  
7         return -1;  
8     }  
9     for(int i=0; i < mida; i++){  
10         v2[i] = v[i]*v[i];  
11     }  
12     return v2;  
13 }
```

FUNCIÓ CALCULAR_QUADRATS

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 1:** Fem una funció que rebi per paràmetre el vector original i retorni el vector modificat

```
1 int main(){
2
3     int v[MIDA] = {1,2,3,4,5};
4     // Opcio 1
5     int * v2;
6     print_taula(v, MIDA);
7     v2 = calcular_quadrats(v, MIDA);
8     print_taula(v2, MIDA);
9 }
```

PROGRAMA PRINCIPAL

```
1 int * calcular_quadrats(int * v, int mida){
2     // Reservem la memòria aquí
3     int * v2;
4     v2 = (int *) malloc (mida * sizeof(int));
5     if (v2 == NULL){
6         printf("Error memòria.\n");
7         return -1;
8     }
9     for(int i=0; i < mida; i++){
10         v2[i] = v[i]*v[i];
11     }
12     return v2;
13 }
```

FUNCIÓ CALCULAR_QUADRATS

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 1:** Fem una funció que rebi per paràmetre el vector original i retorni el vector modificat

```
1 int main(){
2
3     int v[MIDA] = {1,2,3,4,5};
4     // Opcio 1
5     int * v2;
6     print_taula(v, MIDA);
7     v2 = calcular_quadrats(v, MIDA);
8     print_taula(v2, MIDA);
9 }
```

PROGRAMA PRINCIPAL

```
1 int * calcular_quadrats(int * v, int mida){
2     // Reservem la memòria aquí
3     int * v2;
4     v2 = (int *) malloc (mida * sizeof(int));
5     if (v2 == NULL){
6         printf("Error memòria.\n");
7         return NULL;
8     }
9     for(int i=0; i < mida; i++){
10         v2[i] = v[i]*v[i];
11     }
12     return v2;
13 }
```

FUNCIÓ CALCULAR_QUADRATS

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 1:** Fem una funció que rebi per paràmetre el vector original i retorni el vector modificat

```
1 int main(){
2
3   int v[MIDA] = {1,2,3,4,5};
4   // Opcio 1
5   int * v2;
6   print_taula(v, MIDA);
7   v2 = calcular_quadrats(v, MIDA);
8   if (v2 == NULL){
9       printf("Error en l'allocatacio de memoria \n");
10      return -1;
11  }
12  print_taula(v2, MIDA);
13 }
```

PROGRAMA PRINCIPAL

```
1 int * calcular_quadrats(int * v, int mida){
2     // Reservem la memòria aquí
3     int * v2;
4     v2 = (int *) malloc (mida * sizeof(int));
5     if (v2 == NULL){
6         printf("Error memòria.\n");
7         return NULL;
8     }
9     for(int i=0; i < mida; i++){
10        v2[i] = v[i]*v[i];
11    }
12    return v2;
13 }
```

FUNCIÓ CALCULAR_QUADRATS

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors $\{1,2,3,4,5\}$ i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 2:** Fem una acció que rebi per paràmetre el vector original, un vector buit, i en modifiqui aquest segon.

Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 2:** Fem una acció que rebi per paràmetre el vector original, un vector buit, i en modifiqui aquest segon.

```
1 int main(){
2
3     int v[MIDA] = {1,2,3,4,5};
4
5     // Opcio 2
6     int * v2;
7     v2 = (int *) malloc (MIDA * sizeof(int));
8     if (v2 == NULL){
9         printf("Error memòria.\n");
10        return -1;
11    }
12    calcular_quadrats(v, v2, MIDA);
13    print_taula(v2, MIDA);
14 }
```


Procediments que modifiquen taules: acció o funció?

Imaginem que tenim una taula original, de valors {1,2,3,4,5} i hem de crear un procediment que retorni un vector amb els quadrats d'aquests valors.

- **Opció 2:** Fem una acció que rebi per paràmetre el vector original, un vector buit, i en modifiqui aquest segon.

```
1 int main(){
2
3   int v[MIDA] = {1,2,3,4,5};
4
5   // Opcio 2
6   int * v2;
7   v2 = (int *) malloc (MIDA * sizeof(int));
8   if (v2 == NULL){
9       printf("Error memòria.\n");
10      return -1;
11  }
12  calcular_quadrats(v, v2, MIDA);
13  print_taula(v2, MIDA);
14 }
```

PROGRAMA PRINCIPAL

```
1 void calcula_quadrats (int *v, int *v2, int mida){
2     for(int i=0; i < mida; i++){
3         v2[i] = v[i]*v[i];
4     }
5 }
```

FUNCIÓ CALCULAR_QUADRATS

REPASSEM...

FITXERS

Resum de fitxers

Resum de fitxers

Tipus fitxer

Llibreria a incloure: **stdio.h**

Tipus fitxer: **FILE ***

e.g. **FILE * fp;**

Resum de fitxers

Tipus fitxer

Llibreria a incloure: **stdio.h**

Tipus fitxer: **FILE ***

e.g. **FILE * fp;**

Obrir fitxer

fp = fopen(NOM_FITXER, MODE_OBERTURA)

On MODE_OBERTURA sol ser: “r”, “w”, “a” (n’hi ha més)

Si volem llegir de fitxer, ens hem d’assegurar que existeix:

```
if (fp == NULL) { /*Error*/ }
```

Resum de fitxers

Tipus fitxer

Llibreria a incloure: **stdio.h**

Tipus fitxer: **FILE ***

e.g. **FILE * fp;**

Obrir fitxer

fp = fopen(NOM_FITXER, MODE_OBERTURA)

On MODE_OBERTURA sol ser: “r”, “w”, “a” (n’hi ha més)

Si volem llegir de fitxer, ens hem d’assegurar que existeix:

```
if (fp == NULL) { /*Error*/ }
```

Tancar fitxer

fclose(fp)

Tanquem el fitxer quan ja no el necessitem més.

Si estem escrivint, fins que no tanquem no s’escriu res.

Resum de fitxers

Tipus fitxer

Llibreria a incloure: **stdio.h**

Tipus fitxer: **FILE ***

e.g. **FILE * fp;**

Obrir fitxer

fp = fopen(NOM_FITXER, MODE_OBERTURA)

On MODE_OBERTURA sol ser: “r”, “w”, “a” (n’hi ha més)

Si volem llegir de fitxer, ens hem d’assegurar que existeix:

```
if (fp == NULL) { /*Error*/ }
```

Tancar fitxer

fclose(fp)

Tanquem el fitxer quan ja no el necessitem més.

Si estem escrivint, fins que no tanquem no s’escriu res.

Escriure a fitxer

Hem d’haver obert el fitxer en mode “w” o “a”. Podem escriure amb format:

```
fprintf(fp, “La variable a val %d\n”, a);
```


Resum de fitxers

Tipus fitxer

Llibreria a incloure: **stdio.h**

Tipus fitxer: **FILE ***

e.g. **FILE * fp;**

Obrir fitxer

fp = fopen(NOM_FITXER, MODE_OBERTURA)

On MODE_OBERTURA sol ser: “r”, “w”, “a” (n’hi ha més)

Si volem llegir de fitxer, ens hem d’assegurar que existeix:

```
if (fp == NULL) { /*Error*/ }
```

Tancar fitxer

fclose(fp)

Tanquem el fitxer quan ja no el necessitem més.

Si estem escrivint, fins que no tanquem no s’escriu res.

Escriure a fitxer

Hem d’haver obert el fitxer en mode “w” o “a”. Podem escriure amb format:

```
fprintf(fp, “La variable a val %d\n”, a);
```

Llegir de fitxer

Hem d’haver obert el fitxer en mode “r”.

fgets(str, n, fp): llegeix n caràcters o fins que troba un salt de línia, i ho guarda a l’string str. Compte, es guarda el salt de línia.

Resum de fitxers

Tipus fitxer

Llibreria a incloure: **stdio.h**

Tipus fitxer: **FILE ***

e.g. **FILE * fp;**

Obrir fitxer

fp = fopen(NOM_FITXER, MODE_OBERTURA)

On MODE_OBERTURA sol ser: "r", "w", "a" (n'hi ha més)

Si volem llegir de fitxer, ens hem d'assegurar que existeix:

```
if (fp == NULL) { /*Error*/ }
```

Tancar fitxer

fclose(fp)

Tanquem el fitxer quan ja no el necessitem més.

Si estem escrivint, fins que no tanquem no s'escriu res.

Escriure a fitxer

Hem d'haver obert el fitxer en mode "w" o "a". Podem escriure amb format:

```
fprintf(fp, "La variable a val %d\n", a);
```

Llegir de fitxer

Hem d'haver obert el fitxer en mode "r".

fgets(str, n, fp): llegeix n caràcters o fins que troba un salt de línia, i ho guarda a l'string str. Compte, es guarda el salt de línia.

fscanf(fp, "%s *%d %d", str, &x): llegeix tota la línia i intenta fer-ho coincidir amb el format. En aquest cas llegeix un string (que guarda a str), un nombre (que llença) i un altre nombre (que guarda a x).

Resum de fitxers

Tipus fitxer

Llibreria a incloure: **stdio.h**

Tipus fitxer: **FILE ***

e.g. **FILE * fp;**

Obrir fitxer

fp = fopen(NOM_FITXER, MODE_OBERTURA)

On MODE_OBERTURA sol ser: "r", "w", "a" (n'hi ha més)

Si volem llegir de fitxer, ens hem d'assegurar que existeix:

```
if (fp == NULL) { /*Error*/ }
```

Tancar fitxer

fclose(fp)

Tanquem el fitxer quan ja no el necessitem més.

Si estem escrivint, fins que no tanquem no s'escriu res.

Escriure a fitxer

Hem d'haver obert el fitxer en mode "w" o "a". Podem escriure amb format:

```
fprintf(fp, "La variable a val %d\n", a);
```

Llegir de fitxer

Hem d'haver obert el fitxer en mode "r".

fgets(str, n, fp): llegeix n caràcters o fins que troba un salt de línia, i ho guarda a l'string str. Compte, es guarda el salt de línia.

fscanf(fp, "%s *%d %d", str, &x): llegeix tota la línia i intenta fer-ho coincidir amb el format. En aquest cas llegeix un string (que guarda a str), un nombre (que llença) i un altre nombre (que guarda a x).

feof(fp): comprova si hem arribat a final de fitxer. Retorna 0 si no hem arribat al final, diferent de 0 si hem arribat al final. Ens serveix per fer bucles.

Resum de fitxers

Tipus fitxer

Llibreria a incloure: **stdio.h**

Tipus fitxer: **FILE ***

e.g. **FILE * fp;**

Obrir fitxer

fp = fopen(NOM_FITXER, MODE_OBERTURA)

On MODE_OBERTURA sol ser: "r", "w", "a" (n'hi ha més)

Si volem llegir de fitxer, ens hem d'assegurar que existeix:

```
if (fp == NULL) { /*Error*/ }
```

Tancar fitxer

fclose(fp)

Tanquem el fitxer quan ja no el necessitem més.

Si estem escrivint, fins que no tanquem no s'escriu res.

Escriure a fitxer

Hem d'haver obert el fitxer en mode "w" o "a". Podem escriure amb format:

```
fprintf(fp, "La variable a val %d\n", a);
```

Llegir de fitxer

Hem d'haver obert el fitxer en mode "r".

fgets(str, n, fp): llegeix n caràcters o fins que troba un salt de línia, i ho guarda a l'string str. Compte, es guarda el salt de línia.

fscanf(fp, "%s *%d %d", str, &x): llegeix tota la línia i intenta fer-ho coincidir amb el format. En aquest cas llegeix un string (que guarda a str), un nombre (que llença) i un altre nombre (que guarda a x).

feof(fp): comprova si hem arribat a final de fitxer. Retorna 0 si no hem arribat al final, diferent de 0 si hem arribat al final. Ens serveix per fer bucles.

rewind(fp): Quan llegim o escrivim un fitxer tenim el cursor a una certa posició. Rewind el torna a col·locar al començament.

REPASSEM...

NOMBRES ALEATORIS

Nombres pseudo-aleatoris

Com funcionen realment?

- Un ordinador no pot generar nombres purament aleatoris, només pot simular aleatorietat.
- Com ho fa?
 - Un generador de nombres pseudo-aleatoris (pseudo-random number generator (PRNG)) és un programa que agafa un nombre inicial (la llavor, **seed**) i li aplica una sèrie d'operacions matemàtiques a aquest nombre per transformar-lo en un altre nombre que aparentment no està relacionat amb la llavor.
 - Llavors agafa aquest nou nombre i li aplica les mateixes operacions matemàtiques per transformar-lo en un nou nombre també aparentment no relacionat amb l'anterior.
 - Si aquestes operacions matemàtiques són suficientment complexes, i les apliquem de manera continuada aconseguim tenir una seqüència de nombres aparentment no relacionats entre ells (pseudo-aleatoris).

Nombres pseudo-aleatoris

Com funcionen realment?

- Un ordinador no pot generar nombres purament aleatoris, només pot simular aleatorietat.
- Com ho fa?
 - Un generador de nombres pseudo-aleatoris (pseudo-random number generator (PRNG)) és un programa que agafa un nombre inicial (la llavor, **seed**) i li aplica una sèrie d'operacions matemàtiques a aquest nombre per transformar-lo en un altre nombre que aparentment no està relacionat amb la llavor.
 - Llavors agafa aquest nou nombre i li aplica les mateixes operacions matemàtiques per transformar-lo en un nou nombre també aparentment no relacionat amb l'anterior.
 - Si aquestes operacions matemàtiques són suficientment complexes, i les apliquem de manera continuada aconseguim tenir una seqüència de nombres aparentment no relacionats entre ells (pseudo-aleatoris).

clara_rand(): Exemple de generador de nombres pseudo-aleatoris poc sofisticat

Operació matemàtica: fer el quadrat, mòdul 1234 i sumar-li 15

Nombres pseudo-aleatoris

Com funcionen realment?

- Un ordinador no pot generar nombres purament aleatoris, només pot simular aleatorietat.
- Com ho fa?
 - Un generador de nombres pseudo-aleatoris (pseudo-random number generator (PRNG)) és un programa que agafa un nombre inicial (la llavor, **seed**) i li aplica una sèrie d'operacions matemàtiques a aquest nombre per transformar-lo en un altre nombre que aparentment no està relacionat amb la llavor.
 - Llavors agafa aquest nou nombre i li aplica les mateixes operacions matemàtiques per transformar-lo en un nou nombre també aparentment no relacionat amb l'anterior.
 - Si aquestes operacions matemàtiques són suficientment complexes, i les apliquem de manera continuada aconseguim tenir una seqüència de nombres aparentment no relacionats entre ells (pseudo-aleatoris).

clara_rand(): Exemple de generador de nombres pseudo-aleatoris poc sofisticat

Operació matemàtica: fer el quadrat, mòdul 1234 i sumar-li 15

clara_srand()

666

LLAVOR

Nombres pseudo-aleatoris

Com funcionen realment?

- Un ordinador no pot generar nombres purament aleatoris, només pot simular aleatorietat.
- Com ho fa?
 - Un generador de nombres pseudo-aleatoris (pseudo-random number generator (PRNG)) és un programa que agafa un nombre inicial (la llavor, **seed**) i li aplica una sèrie d'operacions matemàtiques a aquest nombre per transformar-lo en un altre nombre que aparentment no està relacionat amb la llavor.
 - Llavors agafa aquest nou nombre i li aplica les mateixes operacions matemàtiques per transformar-lo en un nou nombre també aparentment no relacionat amb l'anterior.
 - Si aquestes operacions matemàtiques són suficientment complexes, i les apliquem de manera continuada aconseguim tenir una seqüència de nombres aparentment no relacionats entre ells (pseudo-aleatoris).

clara_rand(): Exemple de generador de nombres pseudo-aleatoris poc sofisticat

Operació matemàtica: fer el quadrat, mòdul 1234 i sumar-li 15

clara_srand()

666

LLAVOR

565

868

699

1186

Nombres pseudo-aleatoris

clara_rand(): Exemple de generador de nombres pseudo-aleatoris poc sofisticat

Operació matemàtica: fer el quadrat, mòdul 1234 i sumar-li 15

clara_srand()

666

LLAVOR

565

868

699

1186

Quin valor hauria de tenir la llavor?

- Si la llavor té un valor **constant**, la seqüència generada sempre serà la mateixa. (Cada cop que executis el programa tindràs la mateixa seqüència). Útil per tenir un exemple reproducible i trobar possibles problemes.
- Si li donem valors **diferents** a la llavor, les seqüències seran independents entre elles. Per això es fa servir com a llavor l'hora del sistema.

Nombres pseudo-aleatoris

`clararand()`: Exemple de generador de nombres pseudo-aleatoris poc sofisticat

Operació matemàtica: fer el quadrat, mòdul 1234 i sumar-li 15

`clarasrand()`

666

LLAVOR

565

868

699

1186

Quin valor hauria de tenir la llavor?

- Si la llavor té un valor **constant**, la seqüència generada sempre serà la mateixa. (Cada cop que executis el programa tindràs la mateixa seqüència). Útil per tenir un exemple reproducible i trobar possibles problemes.
- Si li donem valors **diferents** a la llavor, les seqüències seran independents entre elles. Per això es fa servir com a llavor l'hora del sistema.

```
1 #include <stdio.h>
2 #include <time.h>
3
4 int main(){
5     printf("Quina hora és? %ld\n", time(NULL));
6     return 0;
7 }
```

Quina hora és? 1670844732

Execució: 12 Dec 2022 12.32h

Nombres pseudo-aleatoris

Per què només hem d'executar **rand un sol cop?**

Imagineu que tinc una funció que genera
seqüències de 5 nombres aleatoris

Nombres pseudo-aleatoris

Per què només hem d'executar `srand` un sol cop?

Imagineu que tinc una funció que genera seqüències de 5 nombres aleatoris

```
1 void genera_seq(){
2     // Què passa si crido a srand aquí dins?
3     srand(time(NULL));
4     printf("Entro a genera_seq\n");
5     for(int i = 0; i < N; i++){
6         printf("%d\n", rand());
7     }
8 }
```


Nombres pseudo-aleatoris

Per què només hem d'executar **srand** un sol cop?

Imagineu que tinc una funció que genera seqüències de 5 nombres aleatoris

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4 #define N 5
5
6 int main(){
7     // Crido a genera_seq
8     genera_seq();
9     return 0;
10 }
```

```
1 void genera_seq(){
2     // Què passa si crido a srand aquí dins?
3     srand(time(NULL));
4     printf("Entro a genera_seq\n");
5     for(int i = 0; i < N; i++){
6         printf("%d\n", rand());
7     }
8 }
```


Nombres pseudo-aleatoris

Per què només hem d'executar **srand** un sol cop?

Imagineu que tinc una funció que genera seqüències de 5 nombres aleatoris

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4 #define N 5
5
6 int main(){
7     // Crido a genera_seq
8     genera_seq();
9     return 0;
10 }
```

```
1 void genera_seq(){
2     // Què passa si crido a srand aquí dins?
3     srand(time(NULL));
4     printf("Entro a genera_seq\n");
5     for(int i = 0; i < N; i++){
6         printf("%d\n", rand());
7     }
8 }
```

Resultat: tot sembla correcte...

```
Entro a genera_seq
211533550
1158939065
612187165
435529378
1317987070
```

Nombres pseudo-aleatoris

Per què només hem d'executar **srand** un sol cop?

Imagineu que tinc una funció que genera seqüències de 5 nombres aleatoris

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4 #define N 5
5
6 int main(){
7     // Crido genera_seq dues vegades
8     genera_seq();
9     genera_seq();
10    return 0;
11 }
```

```
1 void genera_seq(){
2     // Què passa si crido a srand aquí dins?
3     srand(time(NULL));
4     printf("Entro a genera_seq\n");
5     for(int i = 0; i < N; i++){
6         printf("%d\n", rand());
7     }
8 }
```

Nombres pseudo-aleatoris

Per què només hem d'executar **srand** un sol cop?

Imagineu que tinc una funció que genera seqüències de 5 nombres aleatoris

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4 #define N 5
5
6 int main(){
7     // Crido genera_seq dues vegades
8     genera_seq();
9     genera_seq();
10    return 0;
11 }
```

I si crido a la funció `genera_seq()` dos vegades?

```
1 void genera_seq(){
2     // Què passa si crido a srand aquí dins?
3     srand(time(NULL));
4     printf("Entro a genera_seq\n");
5     for(int i = 0; i < N; i++){
6         printf("%d\n", rand());
7     }
8 }
```


Nombres pseudo-aleatoris

Per què només hem d'executar **srand** un sol cop?

Imagineu que tinc una funció que genera seqüències de 5 nombres aleatoris

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4 #define N 5
5
6 int main(){
7     // Crido genera_seq dues vegades
8     genera_seq();
9     genera_seq();
10    return 0;
11 }
```

I si crido a la funció `genera_seq()` dos vegades?

```
1 void genera_seq(){
2     // Què passa si crido a srand aquí dins?
3     srand(time(NULL));
4     printf("Entro a genera_seq\n");
5     for(int i = 0; i < N; i++){
6         printf("%d\n", rand());
7     }
8 }
```

Resultat: les dues seqüències són la mateixa! No hi ha aleatorietat!

```
Entro a genera_seq
217718526
2030615641
746960163
2117542826
1443278498
Entro a genera_seq
217718526
2030615641
746960163
2117542826
1443278498
```

Nombres pseudo-aleatoris

Per què només hem d'executar `srand` un sol cop?

- La funció `srand()` només s'ha de cridar un **sol cop en tota l'execució del programa**
- Si cridem `srand()` a cada funció, estem "resetejant" la llavor al mateix valor (perquè no ha passat ni un segon)
i per tant, les seqüències que obtindrem seran les mateixes!
- On s'ha de fer la crida a `srand()` ?

Nombres pseudo-aleatoris

Per què només hem d'executar **srand** un sol cop?

- La funció `srand()` només s'ha de cridar un **sol cop en tota l'execució del programa**
- Si cridem `srand()` a cada funció, estem "resetejant" la llavor al mateix valor (perquè no ha passat ni un segon) i per tant, les seqüències que obtindrem seran les mateixes!
- On s'ha de fer la crida a `srand()` ?

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4 #define N 5
5
6 int main(){
7     srand(time(NULL));
8     // Crido genera_seq dues vegades
9     genera_seq();
10    genera_seq();
11    return 0;
12 }
```

```
1 void genera_seq(){
2     printf("Entro a genera_seq\n");
3     for(int i = 0; i < N; i++){
4         printf("%d\n", rand());
5     }
6 }
```

Només a la funció main, al començar, i enlloc més

Nombres pseudo-aleatoris

Per què només hem d'executar **srand** un sol cop?

- La funció `srand()` només s'ha de cridar un **sol cop en tota l'execució del programa**
- Si cridem `srand()` a cada funció, estem "resetejant" la llavor al mateix valor (perquè no ha passat ni un segon)
i per tant, les seqüències que obtindrem seran les mateixes!
- On s'ha de fer la crida a `srand()` ?

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4 #define N 5
5
6 int main(){
7     srand(time(NULL));
8     // Crido genera_seq dues vegades
9     genera_seq();
10    genera_seq();
11    return 0;
12 }
```

```
1 void genera_seq(){
2     printf("Entro a genera_seq\n");
3     for(int i = 0; i < N; i++){
4         printf("%d\n", rand());
5     }
6 }
```

```
Entro a genera_seq
1539269743
1918558839
751447368
224585969
1487613204
Entro a genera_seq
1310501254
1002292346
665732154
570511408
70750401
```

Només a la funció `main`, al començar, i enlloc més

REPASSEM...

COMPILACIÓ

Opcions de compilació interessants

Per estar més segurs que el nostre codi no tindrà problemes d'execució...

- Hi ha un mode de compilació que ens dona més warnings dels normals: **-Wall** (enable all warnings)
- A vegades pot ser molt útil per detectar errors
- Us recomano que feu servir aquesta opció per compilar la vostra pràctica.

Opcions de compilació interessants

Per estar més segurs que el nostre codi no tindrà problemes d'execució...

- Hi ha un mode de compilació que ens dona més warnings dels normals: **-Wall** (enable all warnings)
- A vegades pot ser molt útil per detectar errors
- Us recomano que feu servir aquesta opció per compilar la vostra pràctica.

```
1 #include <stdio.h>
2
3 int main(){
4     int i;
5     printf("Num = %d\n", i);
6 }
```

```
iMacClara:Desktop clara$ gcc test.c -o test
```

```
iMacClara:Desktop clara$ gcc test.c -o test -Wall
```

```
test.c:11:26: warning: variable 'i' is uninitialized when used here [-Wuninitialized]
    printf("Num = %d\n", i);
                        ^
```

```
test.c:9:7: note: initialize the variable 'i' to silence this warning
```

```
    int i;
      ^
      = 0
```

```
1 warning generated.
```


Opcions de compilació interessants

Per estar més segurs que el nostre codi no tindrà problemes d'execució...

- **Compte** perquè això no vol dir que tot funcioni bé, aquí per exemple, no donaria cap warning i en canvi el codi no farà el que volem:



```
1 #include <stdio.h>
2
3 void incrementa_num(int *n, int inc){
4     *n = *n + inc;
5 }
6
7 int main(){
8     int i;
9     incrementa_num(i, 10);
10    printf("Num = %d\n", i);
11 }
```

THE END

